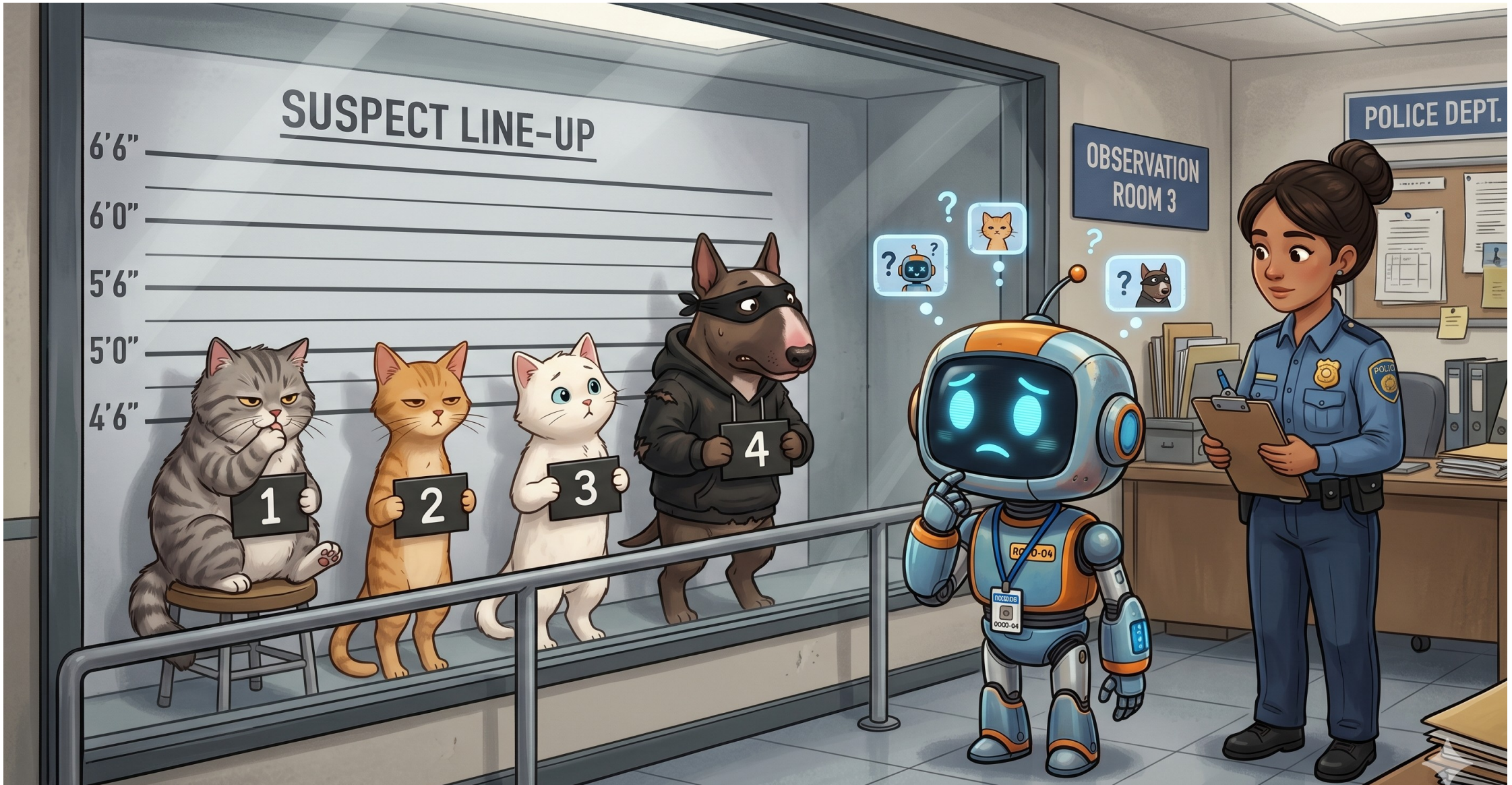
CS-229

Self supervised vision nets intro

Confidence in neural



Inductive biases in vision: invariance and equivariance

Invariance

- A function f[x] is invariant to a transformation t[] if:

$$\mathbf{f}[\mathbf{t}[\mathbf{x}]] = \mathbf{f}[\mathbf{x}]$$

i.e., the function output is the same even after the transformation is applied.

Invariance example

e.g., Image classification

- Image has been translated, but we want our classifier to give the same result

Equivariance

- A function f[x] is equivariant to a transformation t[] if:

$$\mathbf{f}[\mathbf{t}[\mathbf{x}]] = \mathbf{t}[\mathbf{f}[\mathbf{x}]]$$

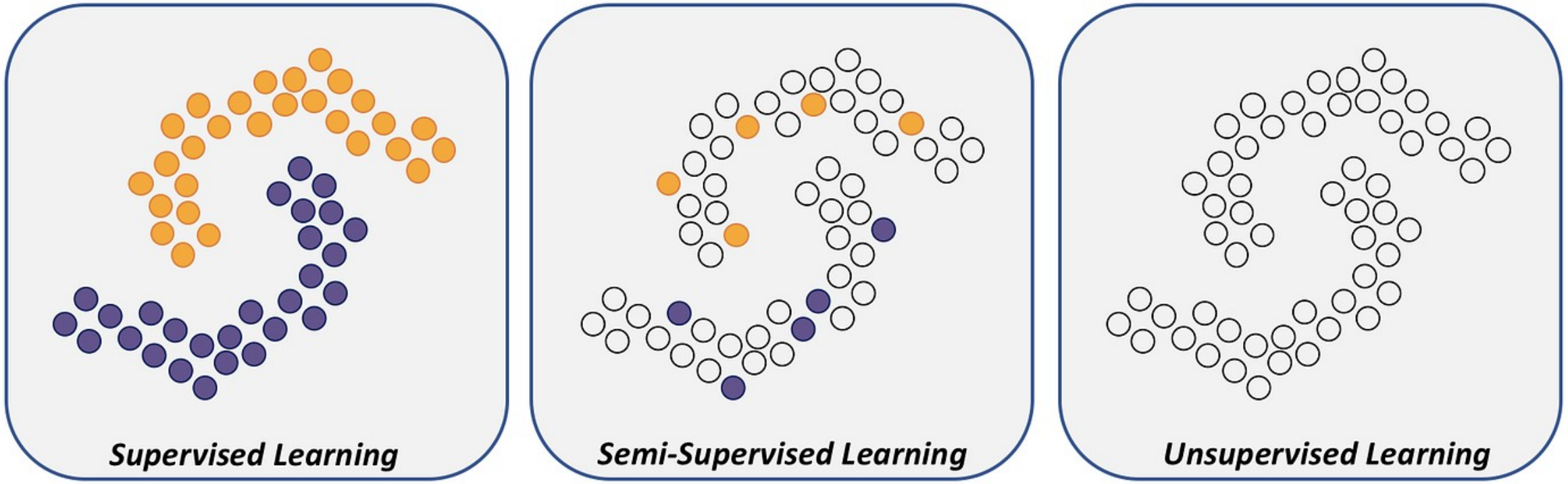
i.e., the output is transformed in the same way as the input

Equivariance example

e.g., Image segmentation

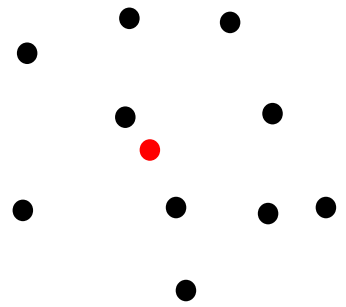
- Image has been translated and we want segmentation to translate with it

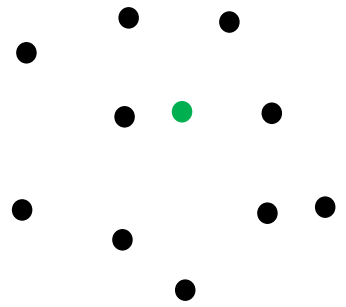
Semi-supervised Vision



How to exploit the large number of unlabeled examples given in semi-supervised learning?

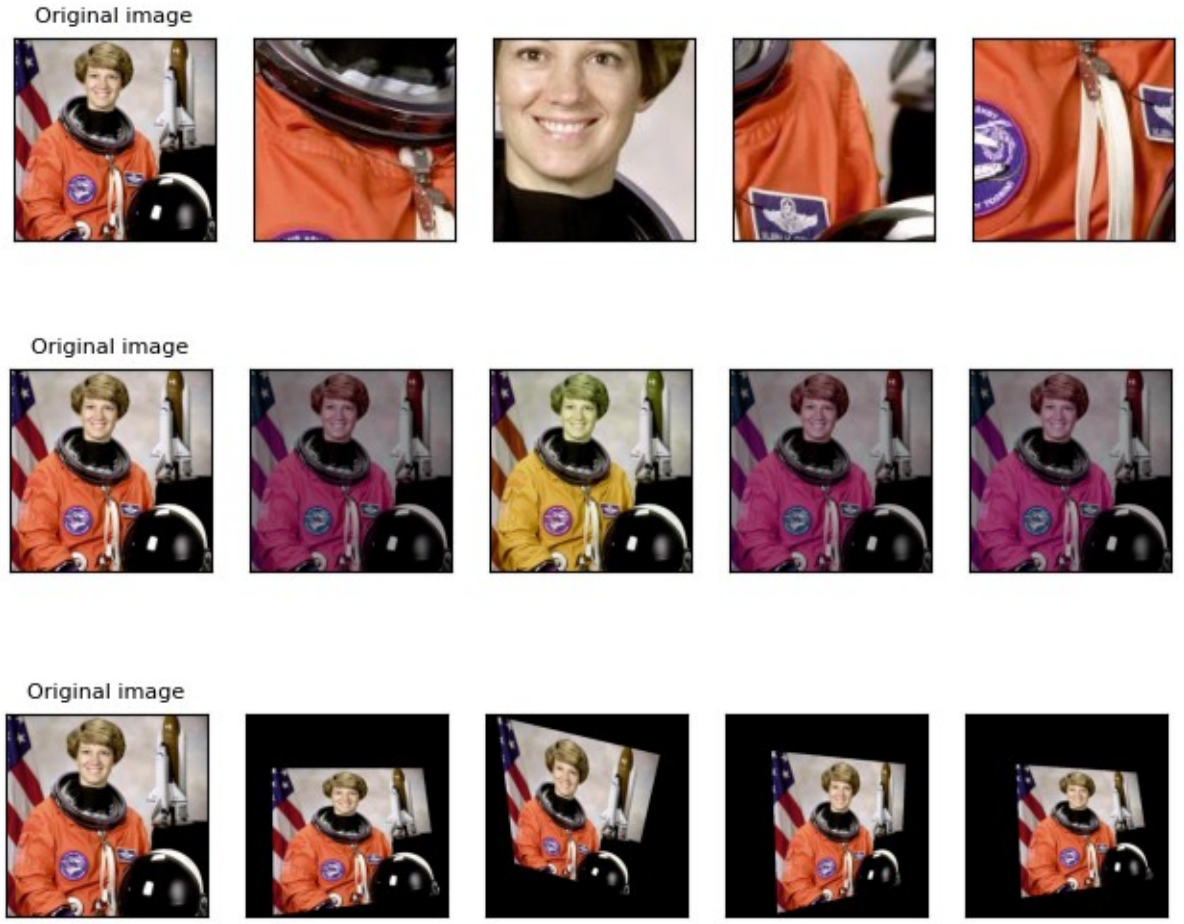
Semi-supervised learning with pseudo labeling and augmenting





Data augmentation in computer vision

https://pytorch.org/vision/stable/transforms.html

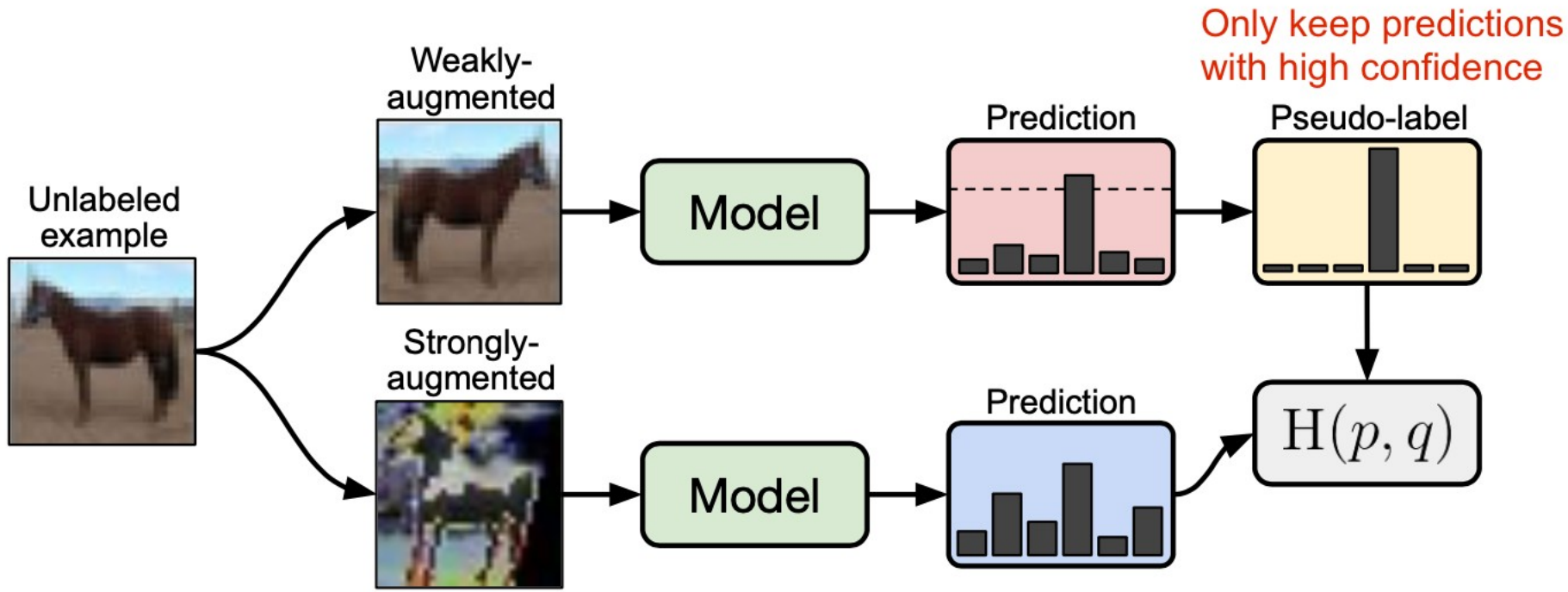


Very useful in self/semi-supervision, but a few problems:

- Hard to replicate this success in non-vision domains
- We should be a little careful, because some augmentation could change the label.

FixMatch: Simplifying Semi-Supervised Learning with Consistency and Confidence

Kihyuk Sohn* **David Berthelot*** **Chun-Liang Li** **Zizhao Zhang** **Nicholas Carlini**
Ekin D. Cubuk **Alex Kurakin** **Han Zhang** **Colin Raffel**
Google Research
{kihyuks,dberth,chunliang,zizhaoz,ncarlini,
cubuk,kurakin,zhanghan,craffel}@google.com



MixMatch: A Holistic Approach to Semi-Supervised Learning

David Berthelot
Google Research
dberth@google.com

Nicholas Carlini
Google Research
ncarlini@google.com

Ian Goodfellow
Work done at Google
ian-academic@mailfence.com

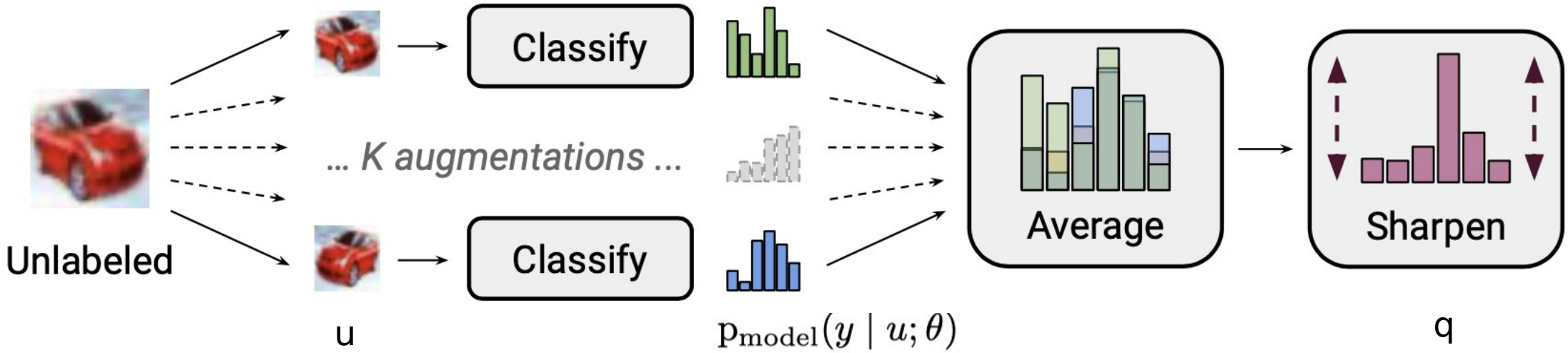
Avital Oliver
Google Research

Nicolas Papernot
Google Research

Colin Raffel
Google Research

Loss = cross entropy (on labeled examples) +

$$\mathcal{L}_{\mathcal{U}} = \frac{1}{L|\mathcal{U}'|} \sum_{u,q \in \mathcal{U}'} \|q - \mathrm{p}_{\mathrm{model}}(y \mid u; \theta)\|_2^2$$



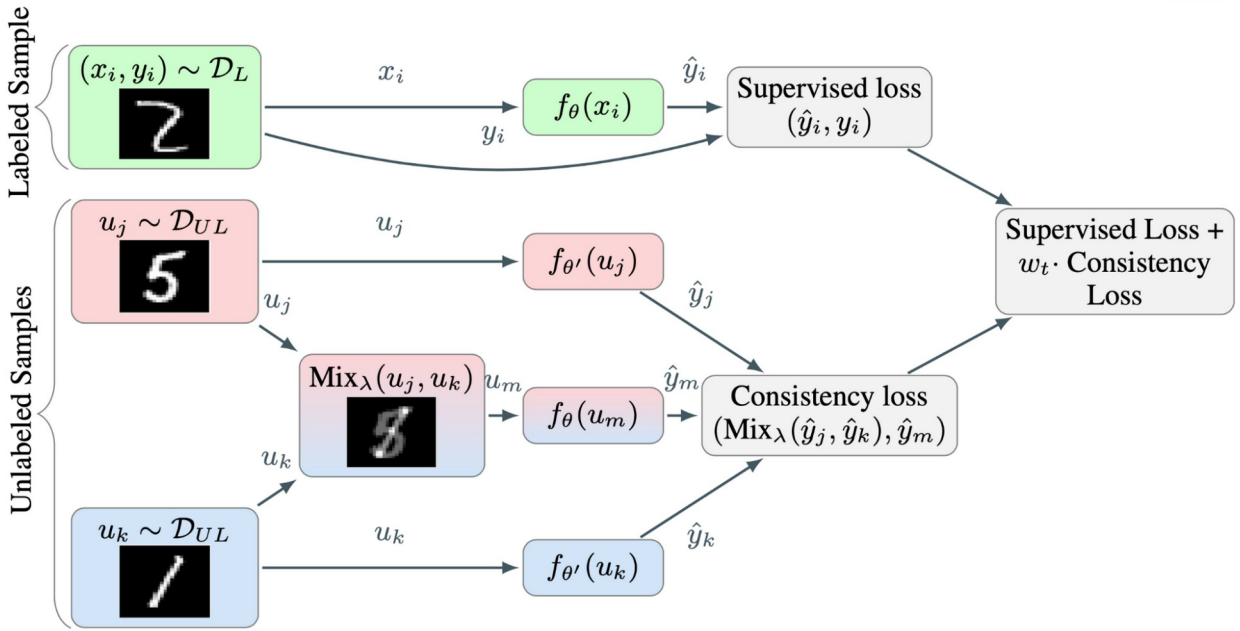
1.Consistency regularization: Encourage the model to output the same predictions on perturbed unlabeled samples.
2.Entropy minimization: Encourage the model to output confident predictions on unlabeled data.
3.MixUp augmentation: (coming up)

mixup: BEYOND EMPIRICAL RISK MINIMIZATION

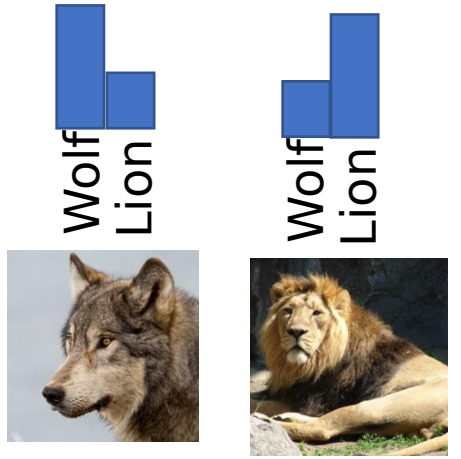
Hongyi Zhang
MIT

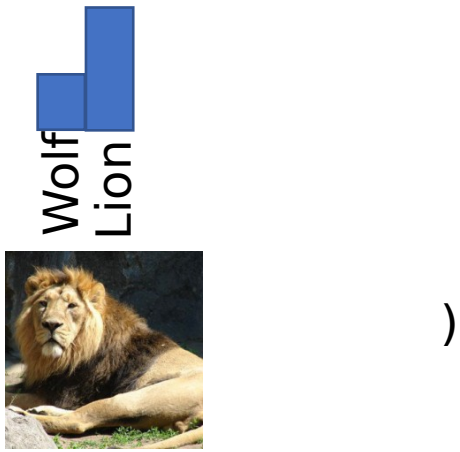
Moustapha Cisse, Yann N. Dauphin, David Lopez-Paz*
FAIR

$$\text{mixup}_\lambda(\mathbf{x}_i, \mathbf{x}_j) = \lambda \mathbf{x}_i + (1-\lambda)\mathbf{x}_j$$
$$p(\text{mixup}_\lambda(y \mid \mathbf{x}_i, \mathbf{x}_j)) \approx \lambda p(y \mid \mathbf{x}_i) + (1-\lambda) p(y \mid \mathbf{x}_j)$$



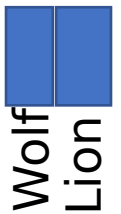
Mixup(





)

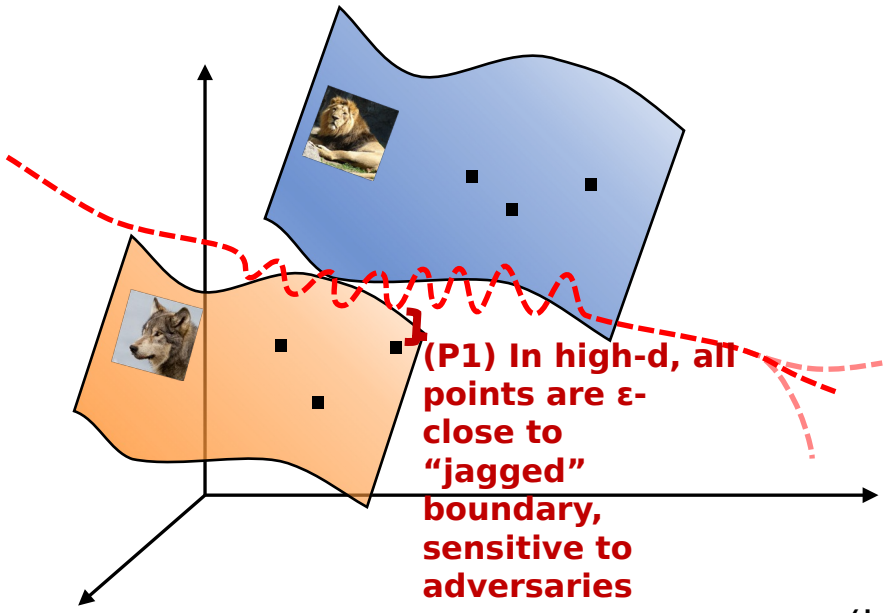
=



mixup: BEYOND EMPIRICAL RISK MINIMIZATION

Hongyi Zhang
MIT

Moustapha Cisse, Yann N. Dauphin, David Lopez-Paz*
FAIR



My interpretation – mixup constrains the shape of the classifier between these manifolds

Improves adversary robustness and confidence calibration too (Wednesday?)

Self-supervised Computer Vision

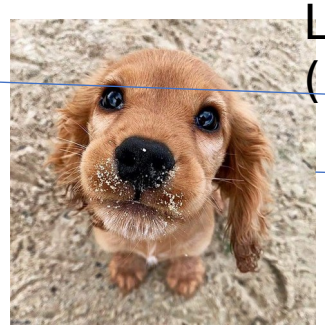
Maybe you can’t pretrain on a similar large labeled dataset.

Then you should consider pretraining on a large *self-supervised* task

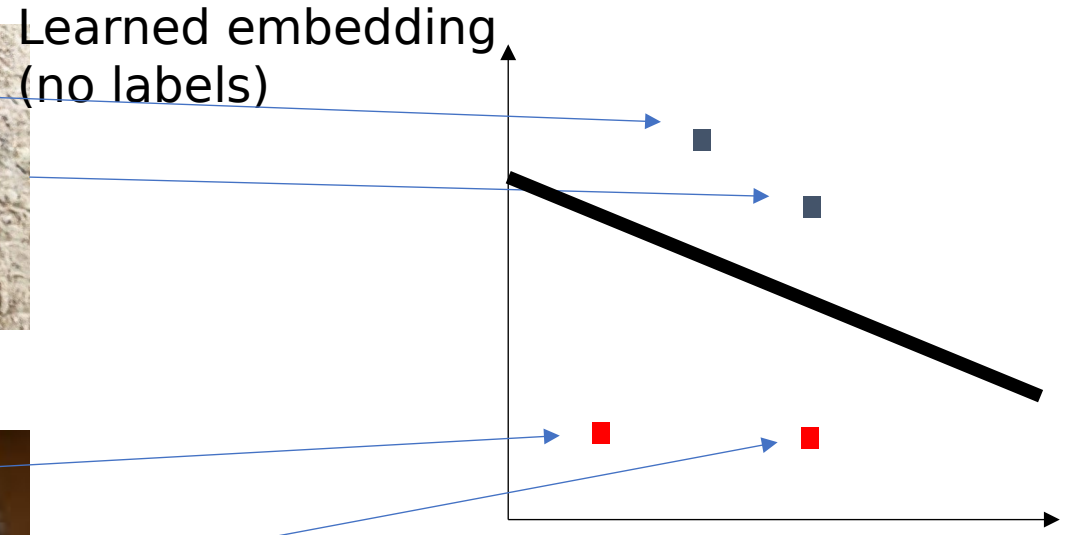
We know what to do for LLMS (next word prediction), what do we do in vision?

Why/when does learning a self-supervised embedding help?

X

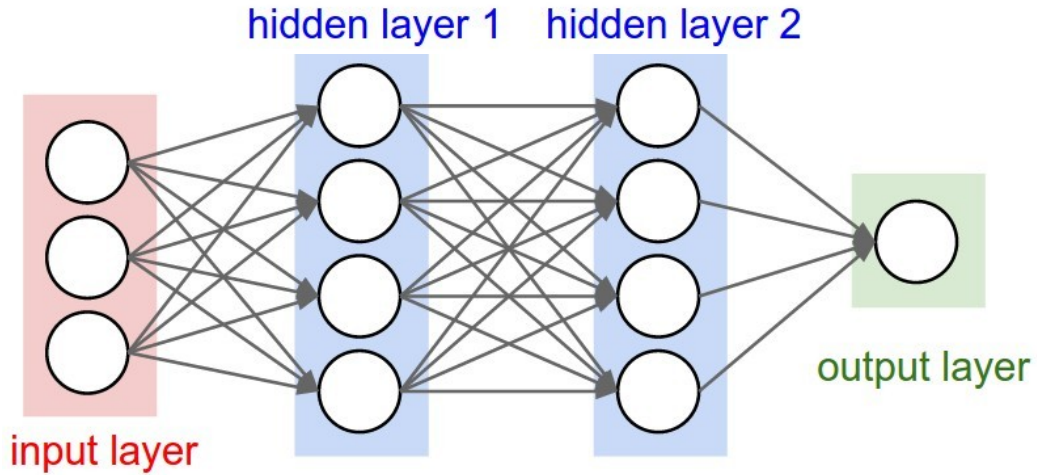


Z



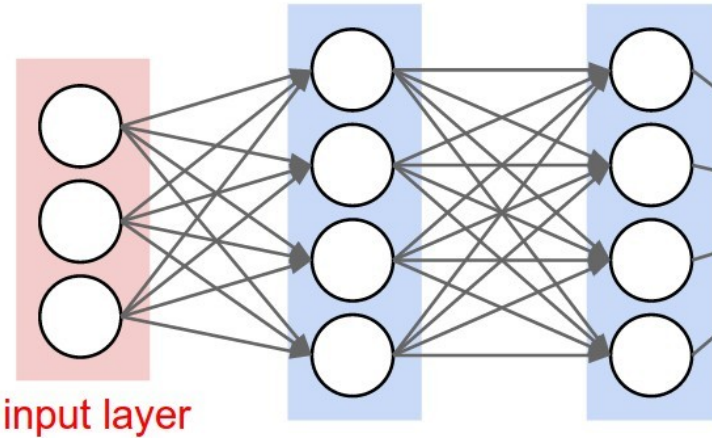
Classes are linearly separable in learned space
(If we picked a good pretext task!)

Self-supervised pipeline evaluation



Self supervised, or “pre-text” classification task

(1) Self-supervised training



$\hat{y} = W\,z + b$

Train linear model using z, to predict target task

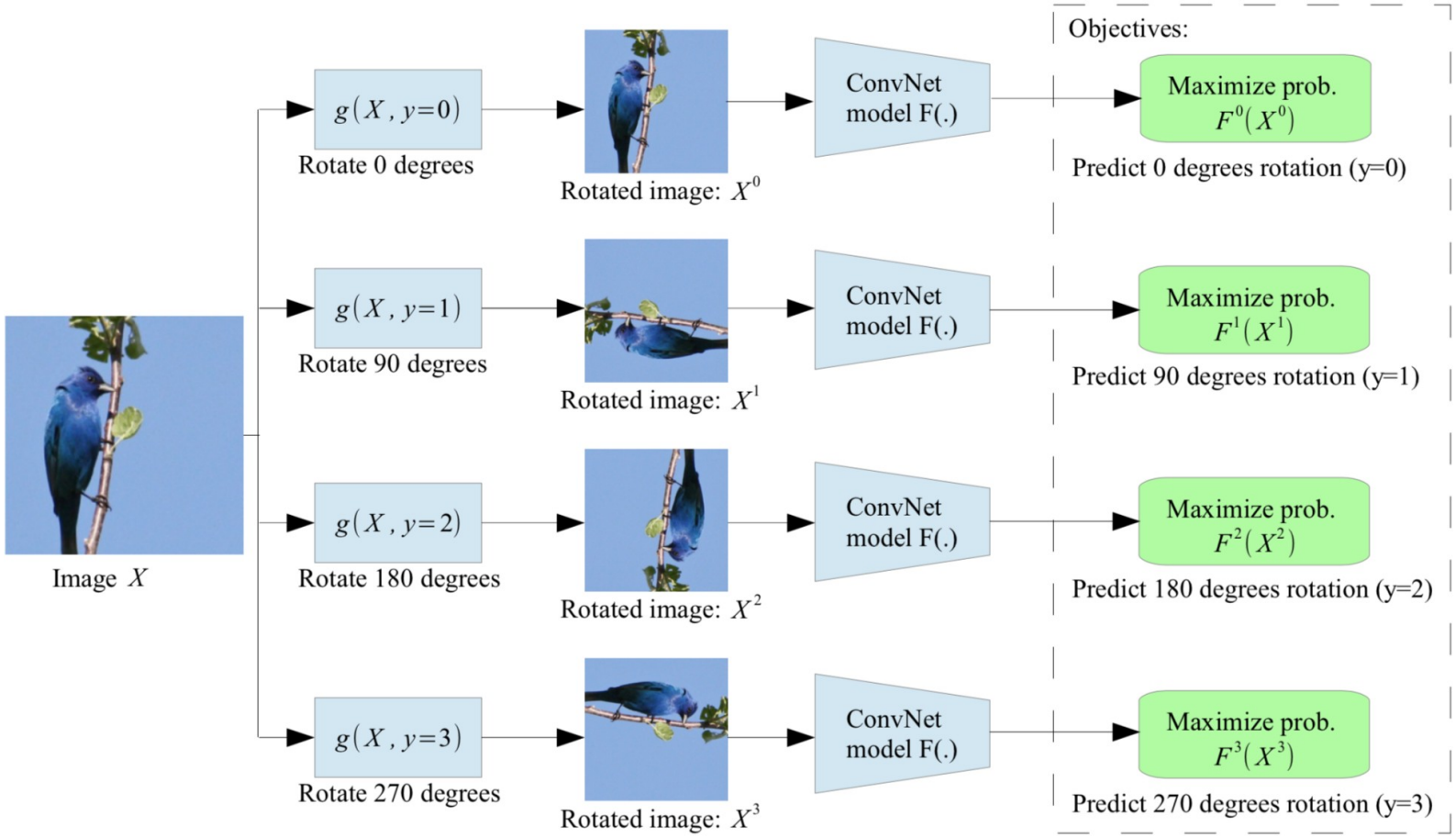
(2) Transfer to target task

z

This is how self-supervised methods are evaluated in literature

History of things people tried

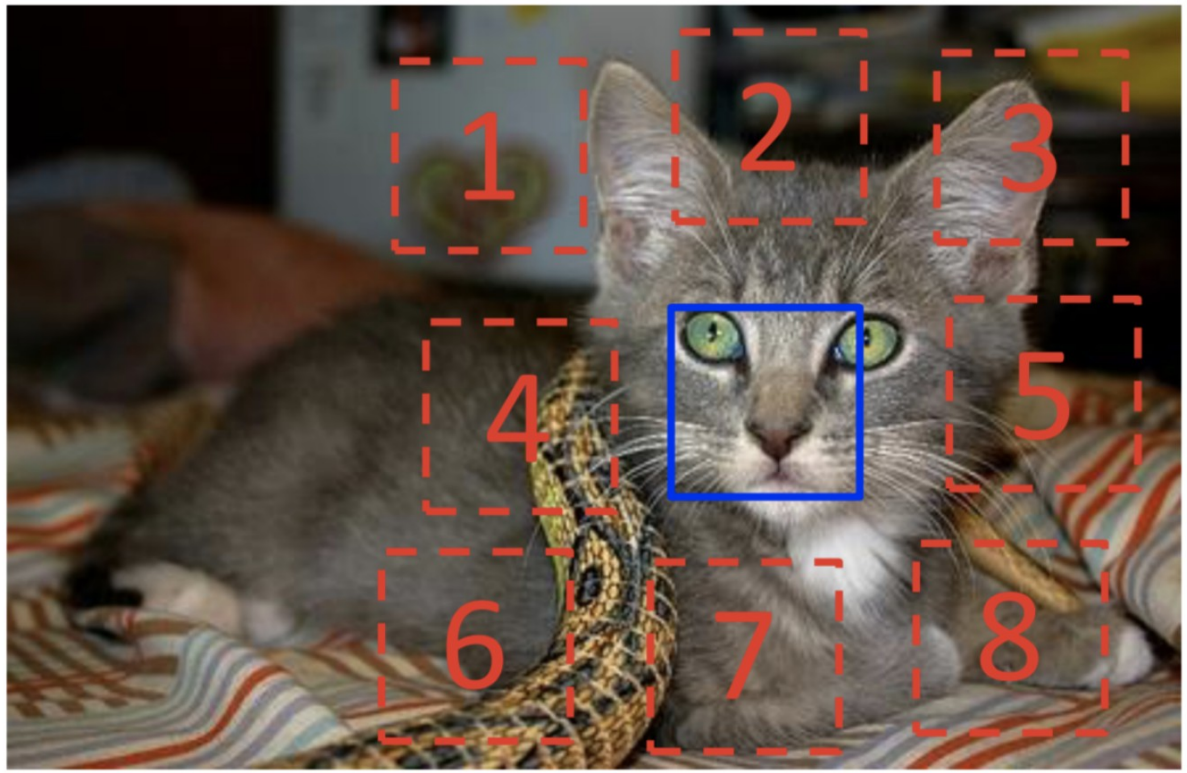
Predict rotations!



Unsupervised Representation Learning by Predicting Image Rotations

Spyros Gidaris, Praveer Singh, Nikos Komodakis

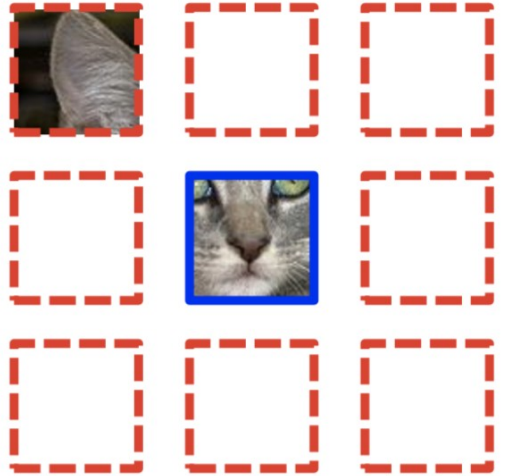
Predict context



X = ( , ); Y = 3

Example:





Question 1:

?

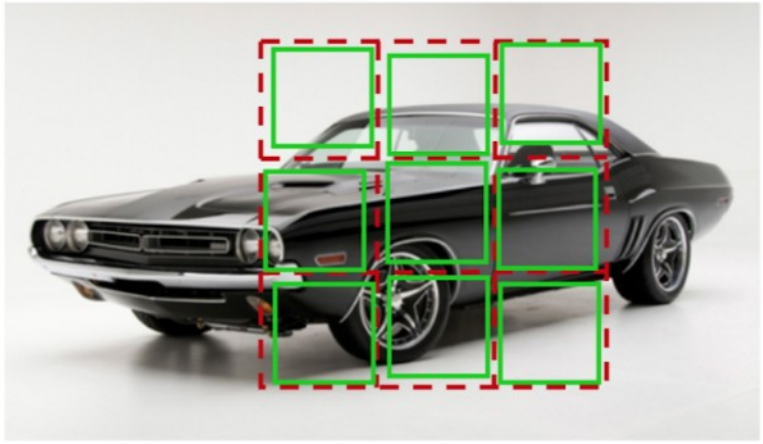
Question 2:

?

Unsupervised Visual Representation Learning by Context Prediction

Carl Doersch, Abhinav Gupta, Alexei A. Efros

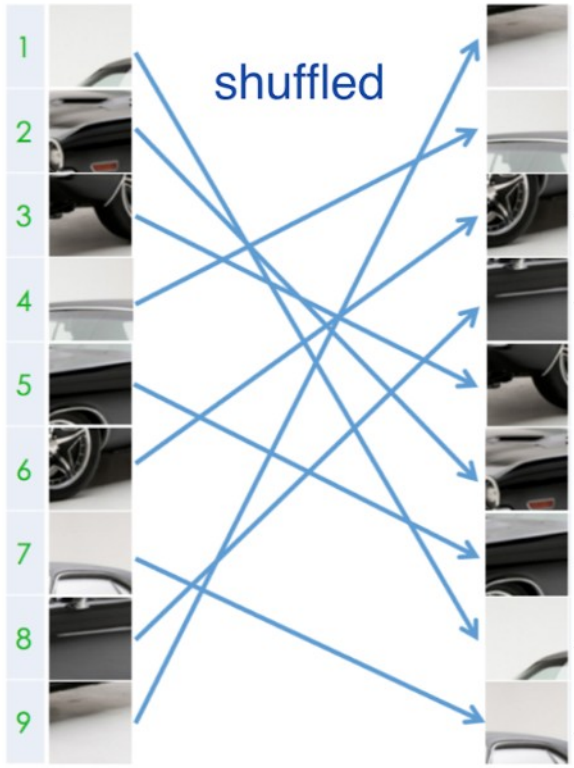
Solve puzzles

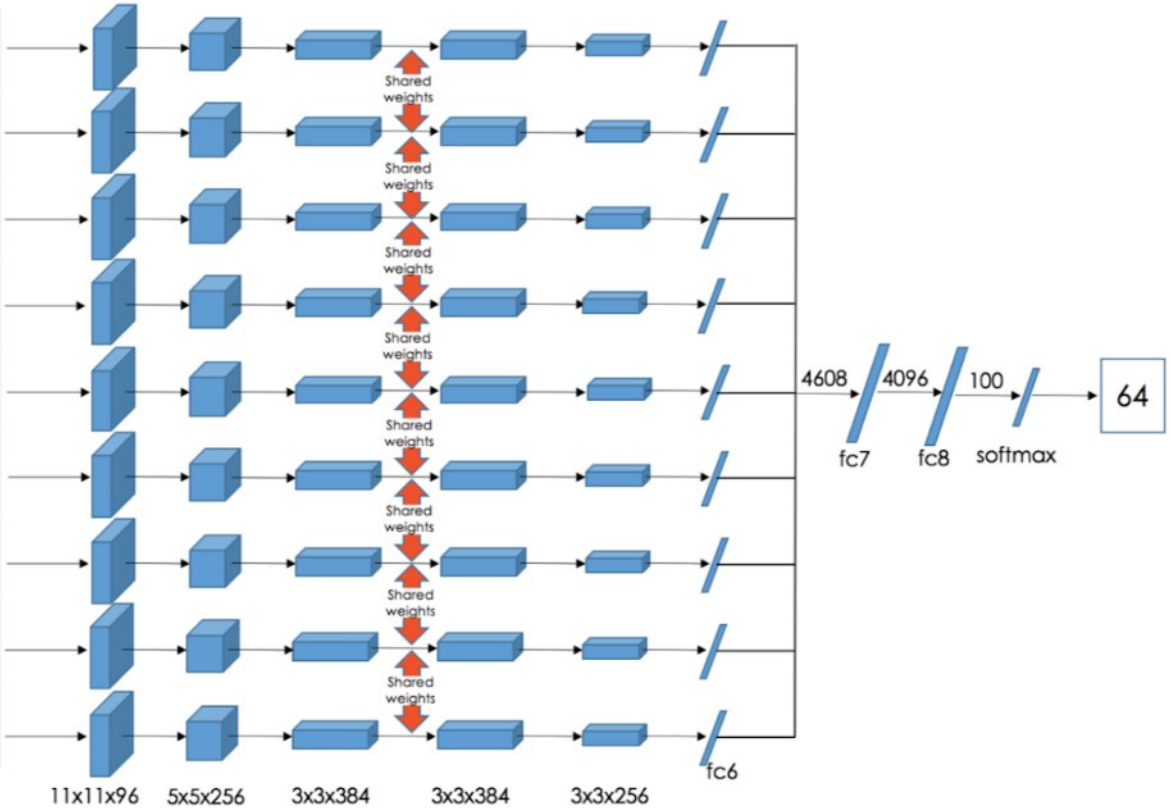


Permutation Set

index	permutation
64	9,4,6,8,3,2,5,1,7

Reorder patches according to the selected permutation

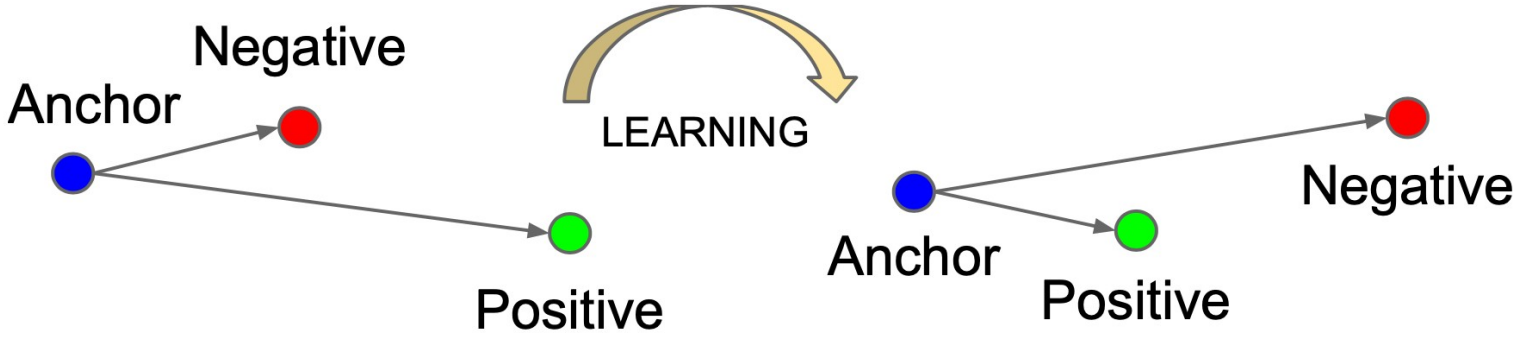


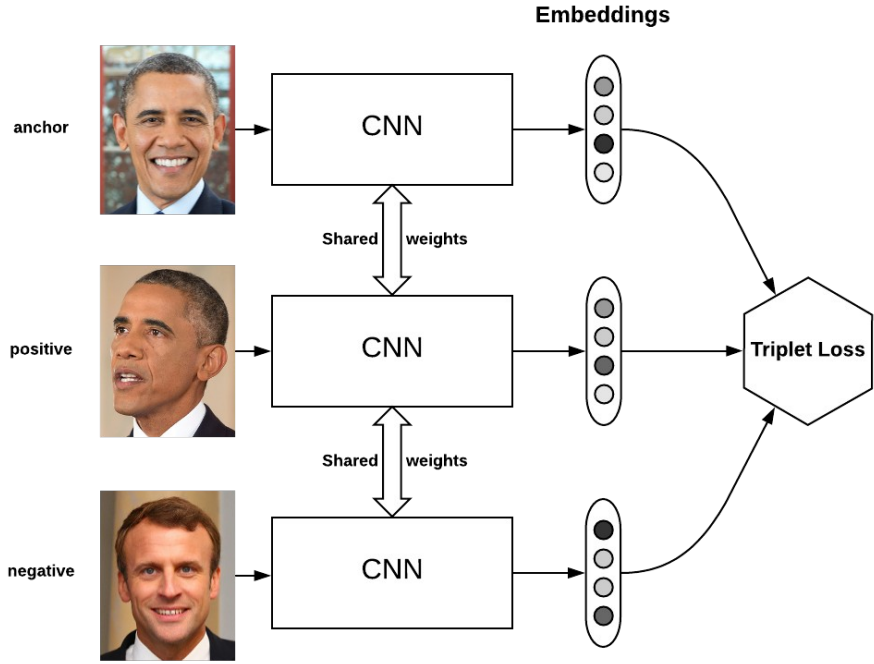


Unsupervised Learning of Visual Representations by Solving Jigsaw Puzzles

Mehdi Noroozi, Paolo Favaro

Contrastive learning – triplet loss





$$\mathcal{L}_{\text{triplet}}(\mathbf{x}, \mathbf{x}^+, \mathbf{x}^-) = \sum_{\mathbf{x} \in \mathcal{X}} \max\left(0, \|f(\mathbf{x}) - f(\mathbf{x}^+)\|_2^2 - \|f(\mathbf{x}) - f(\mathbf{x}^-)\|_2^2 + \epsilon\right)$$

Make these closeMake these far apart

To do it in a more self-supervised way, the positive could be a modification of the anchor image, and the

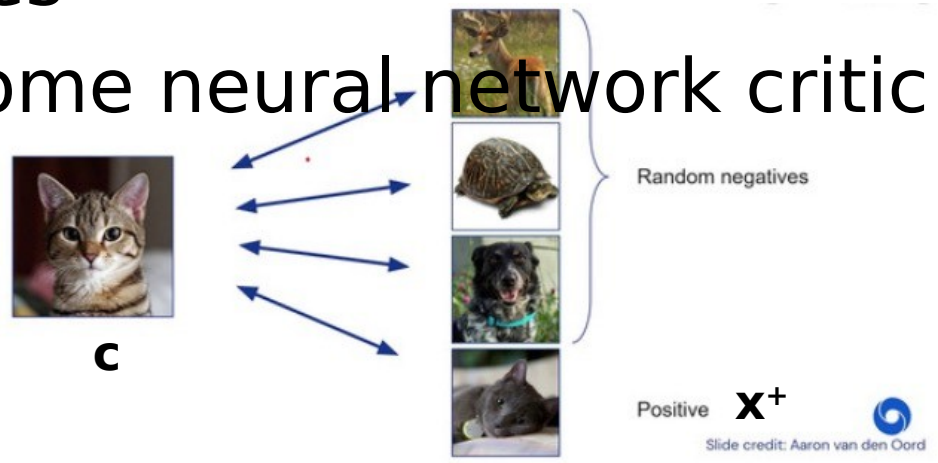
More popular/modern form of contrastive loss: InfoNCE / CPC

"Info Noise Contrastive Estimation", "Contrastive Predictive Coding"

Original paper: https://arxiv.org/abs/1807.03748

Predict a match in a batch with one positive and many negatives

Using some neural network critic function, *f*, a classifier



$$\mathcal{L}_{\text{InfoNCE}} = -\mathbb{E}\left[\log \frac{f(\mathbf{x}^+, \mathbf{c})}{\sum_{\mathbf{x}' \in X} f(\mathbf{x}', \mathbf{c})}\right]$$

(Often something like cosine similarity between

Contrastive loss math

We want to define a probability distribution – predict which of the images on the right is correctly matched with left.

First, use a similarity measure. Dot product is common:

Sim(a, i) = $z_a \cdot z_j$

How do we turn it into a distribution?

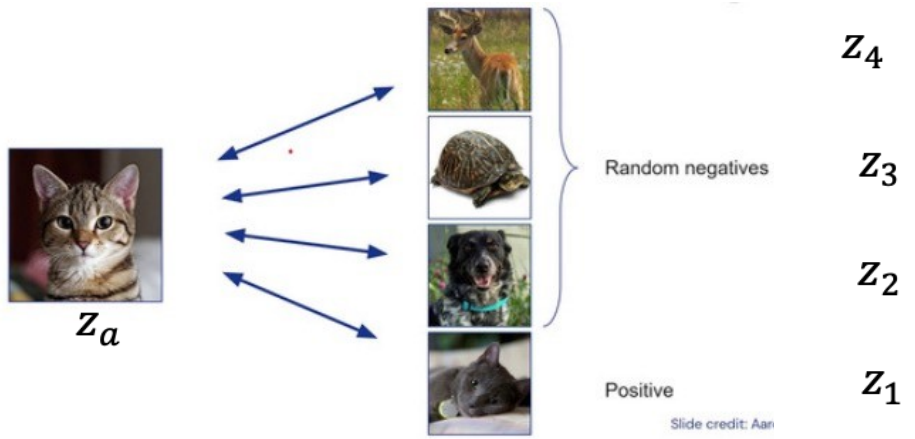
$$p(y=j) = softmax\ sim(z_a, z_j) = \frac{e^{z_a \cdot z_j}}{\sum_i e^{z_a \cdot z_i}}$$

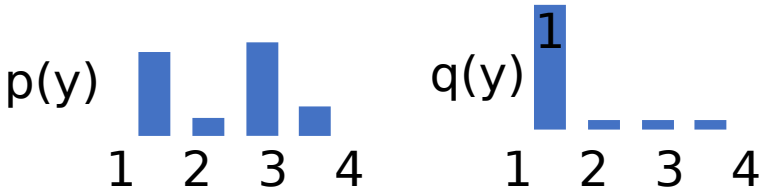
Now, we have to compare this to the *true* answer.

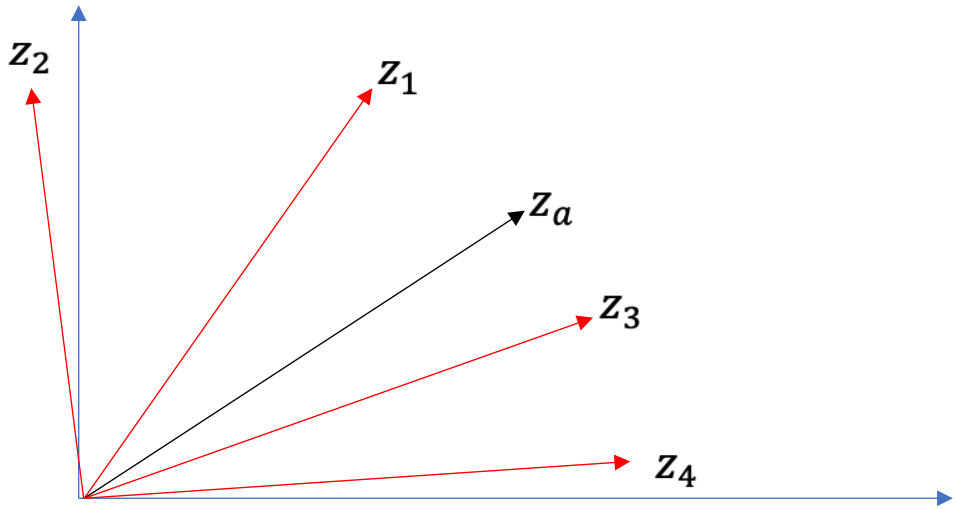
We have set it in this case, y=1, or q(y=1) = 1, q(y≠1) = 0.

Cross entropy:

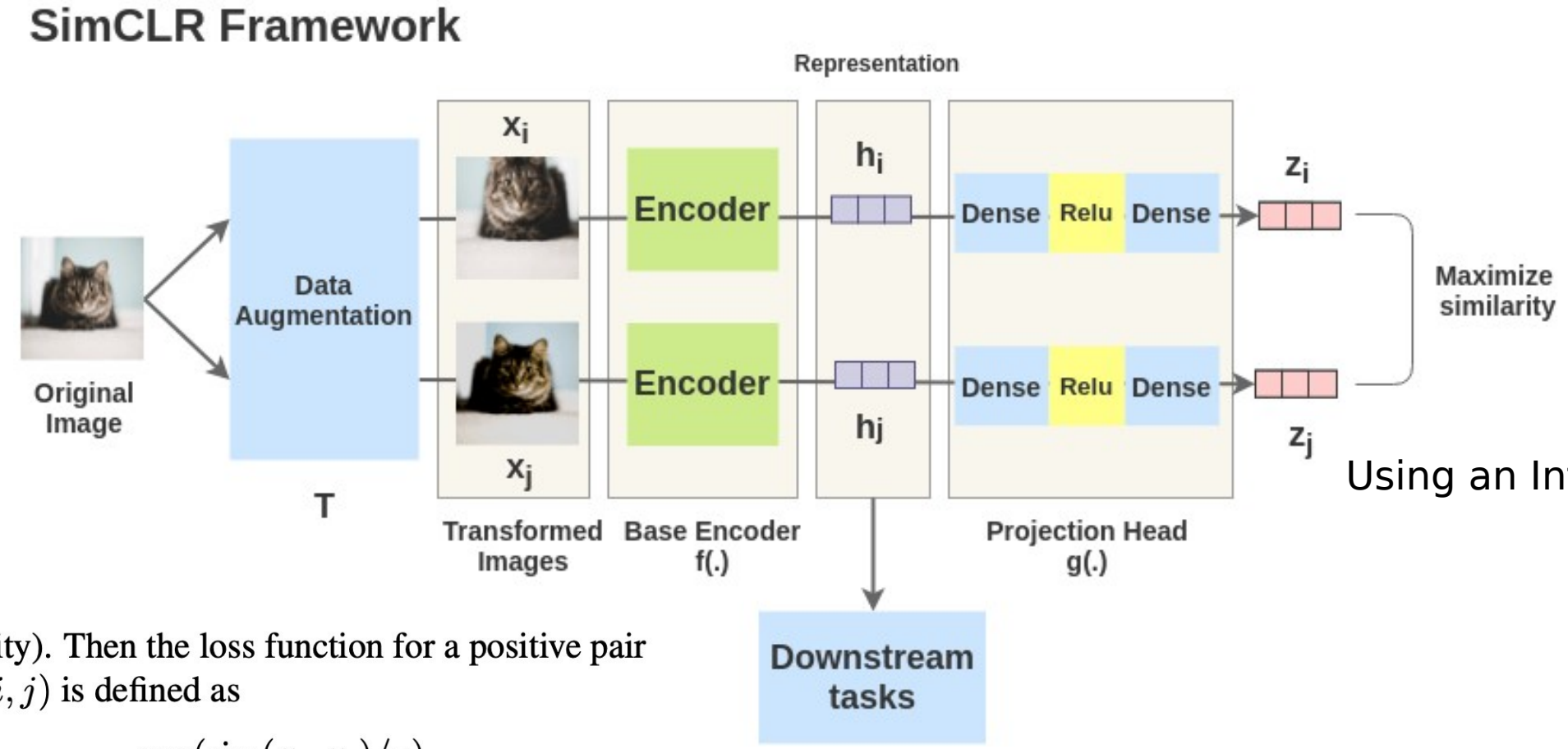
$$H(p,q) = -\sum_y q(y) \log p(y) = -\log \frac{e^{z_a \cdot z_1}}{\sum_i e^{z_a \cdot z_i}}$$







SimCLR – Self-supervised contrastive similarity learning

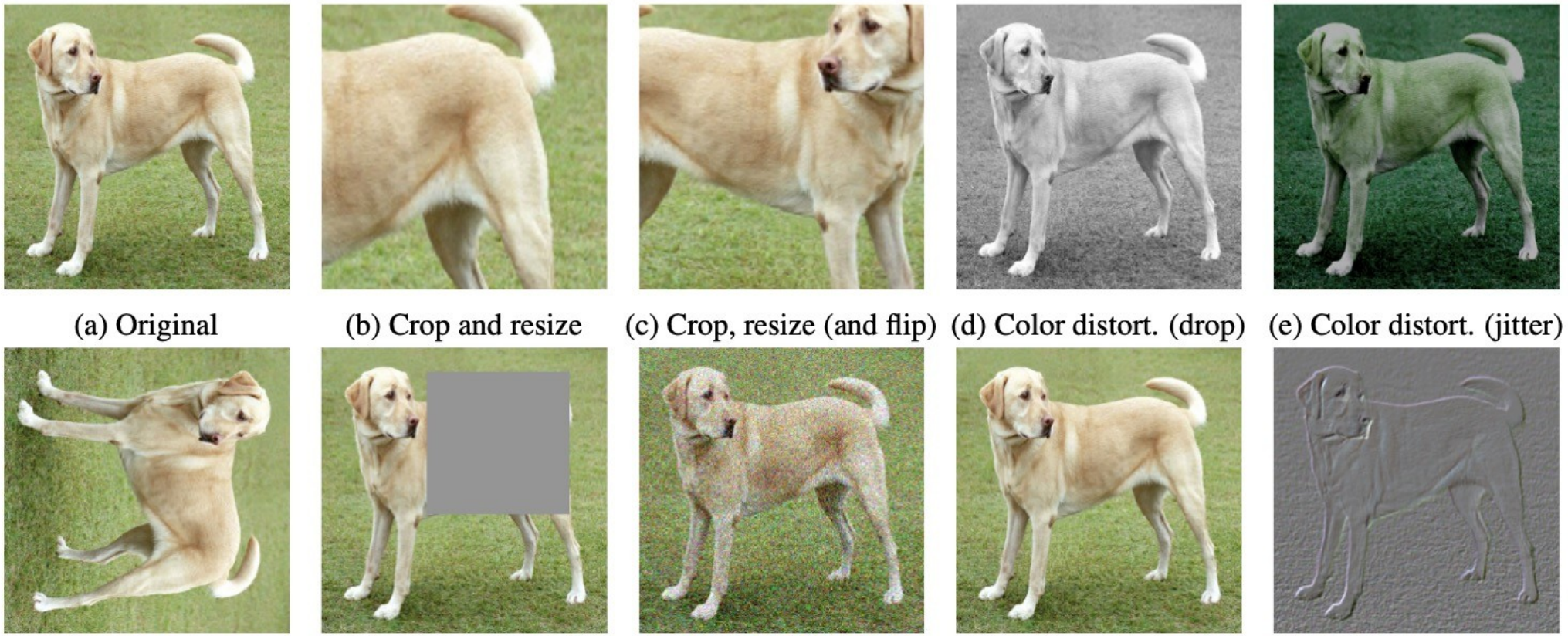


Using an InfoNCE style los

cosine similarity). Then the loss function for a positive pair of examples (i, j) is defined as

$$\ell_{i,j} = -\log \frac{\exp(\text{sim}(\boldsymbol{z}_i, \boldsymbol{z}_j)/\tau)}{\sum_{k=1}^{2N} \mathbb{1}_{[k \neq i]} \exp(\text{sim}(\boldsymbol{z}_i, \boldsymbol{z}_k)/\tau)}, \quad (1)$$

SimCLR augmentations



(a) Original (b) Crop and resize (c) Crop, resize (and flip) (d) Color distort. (drop) (e) Color distort. (jitter)

(f) Rotate $\{90°, 180°, 270°\}$ (g) Cutout (h) Gaussian noise (i) Gaussian blur (j) Sobel filtering

SimCLRv2

Big Self-Supervised Models are Strong Semi-Supervised Learners

Ting Chen, Simon Kornblith, Kevin Swersky, Mohammad Norouzi, Geoffrey Hinton
Google Research, Brain Team

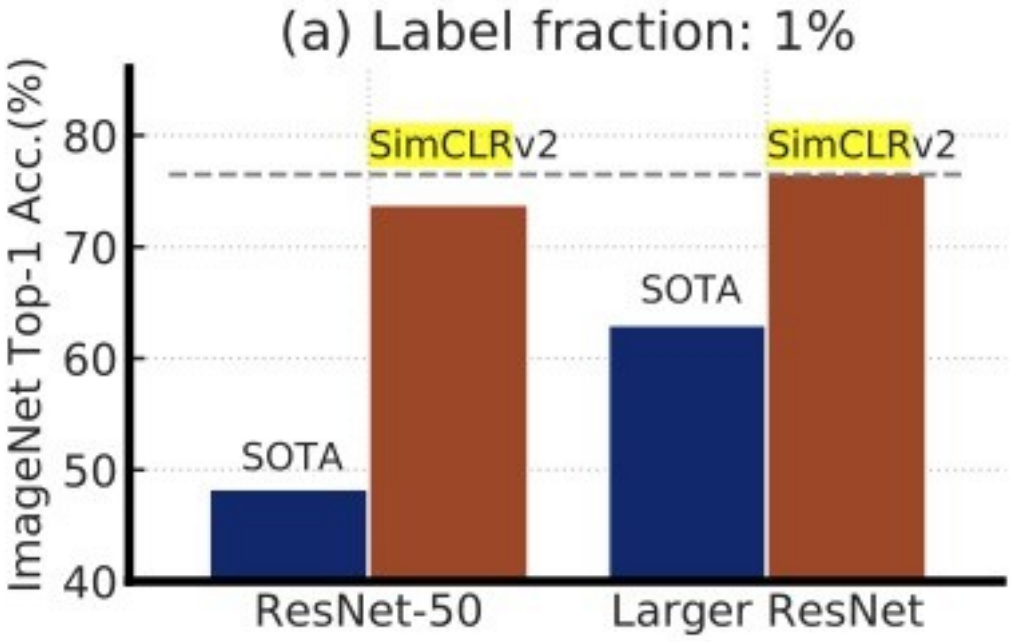


Figure 2: Top-1 accuracy of previous state-of-the-art (SOTA) methods [1, 2] and our method (SimCLRv2) on ImageNet using only 1% or 10% of the labels. Dashed line denotes fully supervised ResNet-50 trained with 100% of labels. Full comparisons in Table 3.

Quick aside

CLIP: **Contrastive Language–Image Pretraining**

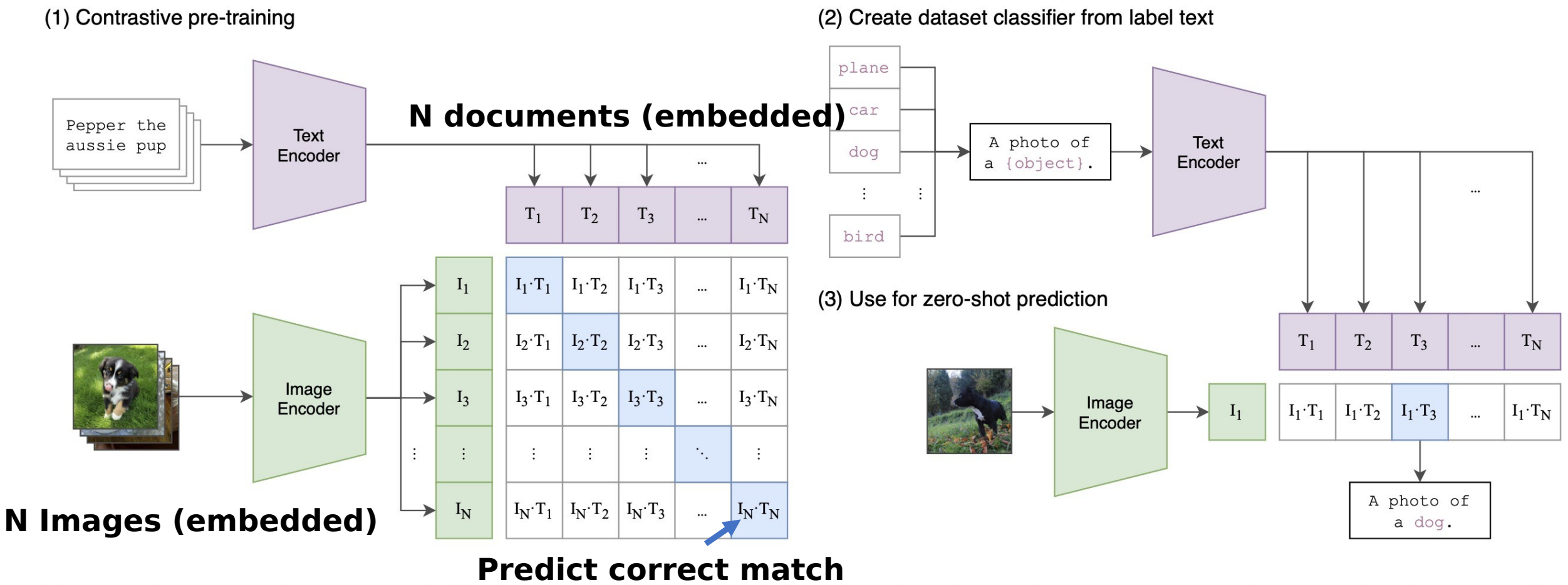
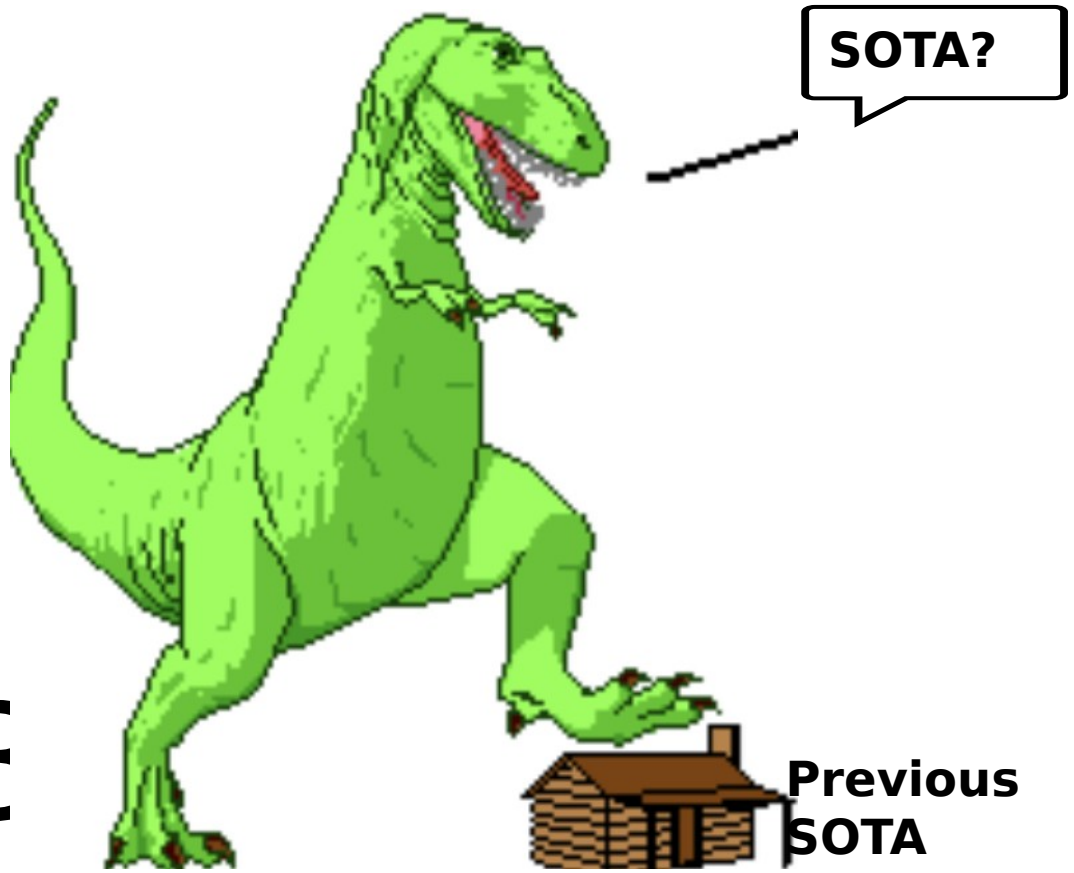


Figure 1. Summary of our approach. While standard image models jointly train an image feature extractor and a linear classifier to predict some label, CLIP jointly trains an image encoder and a text encoder to predict the correct pairings of a batch of (image, text) training examples. At test time the learned text encoder synthesizes a zero-shot linear classifier by embedding the names or descriptions of the target dataset's classes.

```
# image_encoder - ResNet or Vision Transformer
# text_encoder  - CBOW or Text Transformer
# I[n, h, w, c] - minibatch of aligned images
# T[n, l]       - minibatch of aligned texts
# W_i[d_i, d_e] - learned proj of image to embed
# W_t[d_t, d_e] - learned proj of text to embed
# t             - learned temperature parameter

# extract feature representations of each modality
I_f = image_encoder(I) #[n, d_i]
T_f = text_encoder(T)  #[n, d_t]

# joint multimodal embedding [n, d_e]
I_e = l2_normalize(np.dot(I_f, W_i), axis=1)
T_e = l2_normalize(np.dot(T_f, W_t), axis=1)

# scaled pairwise cosine similarities [n, n]
logits = np.dot(I_e, T_e.T) * np.exp(t)

# symmetric loss function
labels = np.arange(n)
loss_i = cross_entropy_loss(logits, labels, axis=0)
loss_t = cross_entropy_loss(logits, labels, axis=1)
loss   = (loss_i + loss_t)/2
```

Figure 3. Numpy-like pseudocode for the core of an implementation of CLIP.

DINO (v1-v3



DINO v1: Self-**DI**stillation w **NO** Labels

Vision transformer backbone – attention learns about objects without labels!

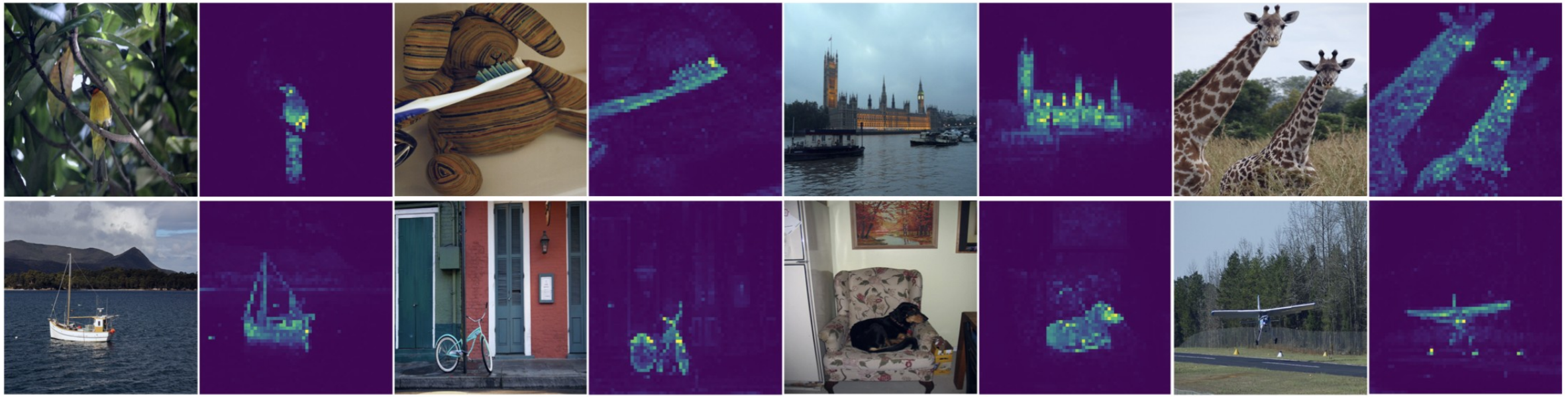
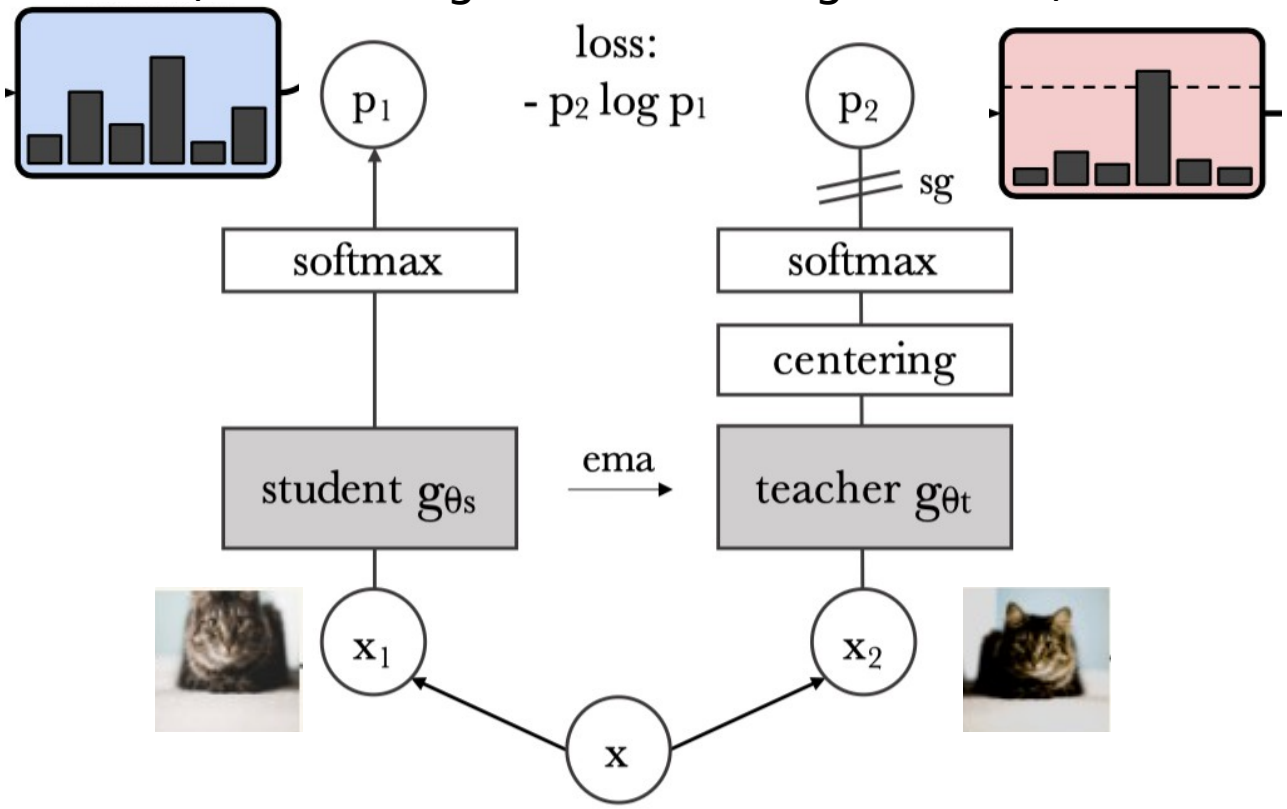


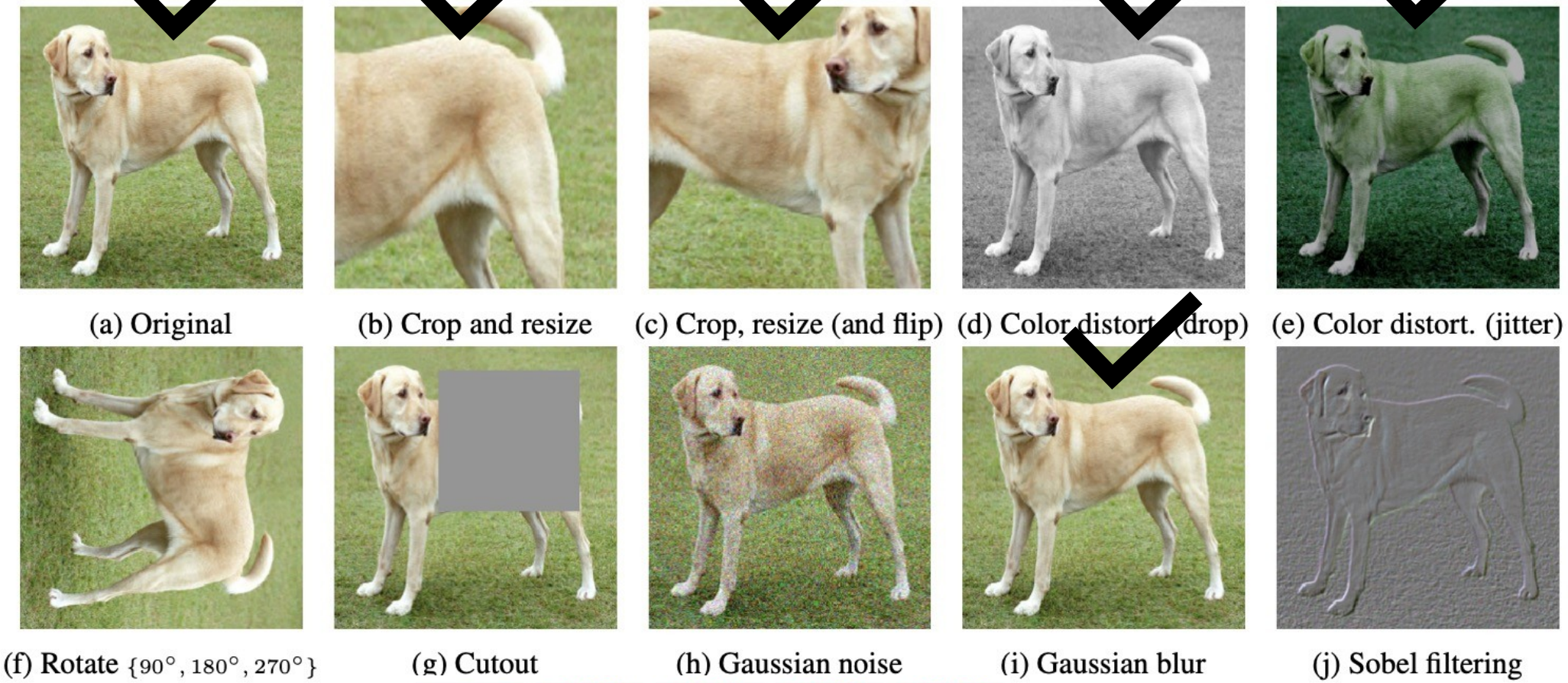
Figure 1: **Self-attention from a Vision Transformer with** 8×8 **patches trained with no supervision.** We look at the self-attention of the `[CLS]` token on the heads of the last layer. This token is not attached to any label nor supervision. These maps show that the model automatically learns class-specific features leading to unsupervised object segmentations.

Predict same “label” distribution
(even though we have no g.t. labels)



Augmented views just like before

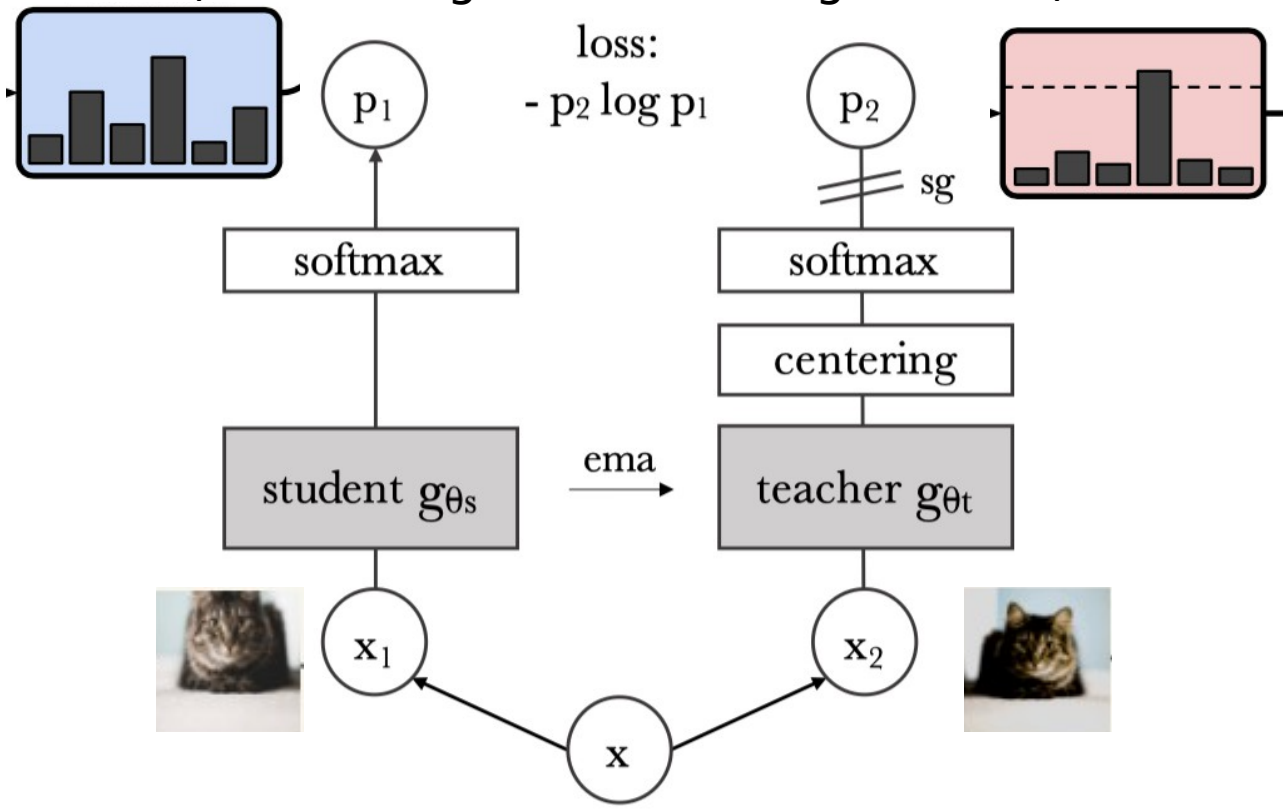
Mostly same augmentation as SimCLR



+”solarization”

“self-distillation”? Teacher = Exponential Moving Average of Student?

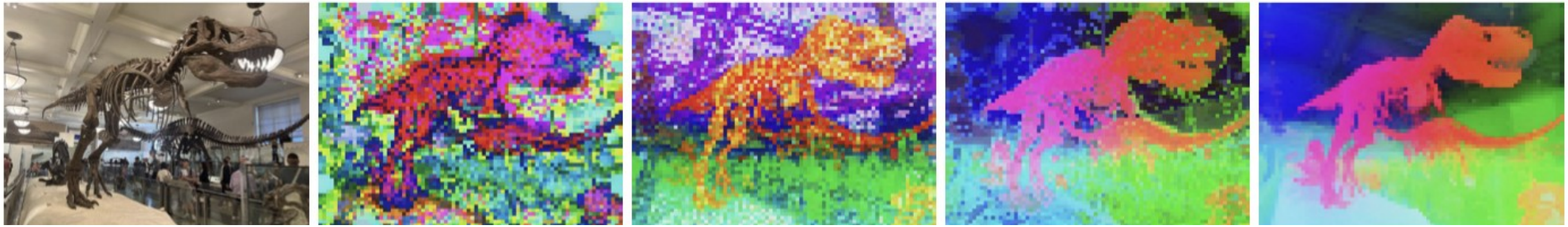
Predict same “label” distribution
(even though we have no g.t. labels)



Augmented views just like before

v2-3: mostly scaling improvements

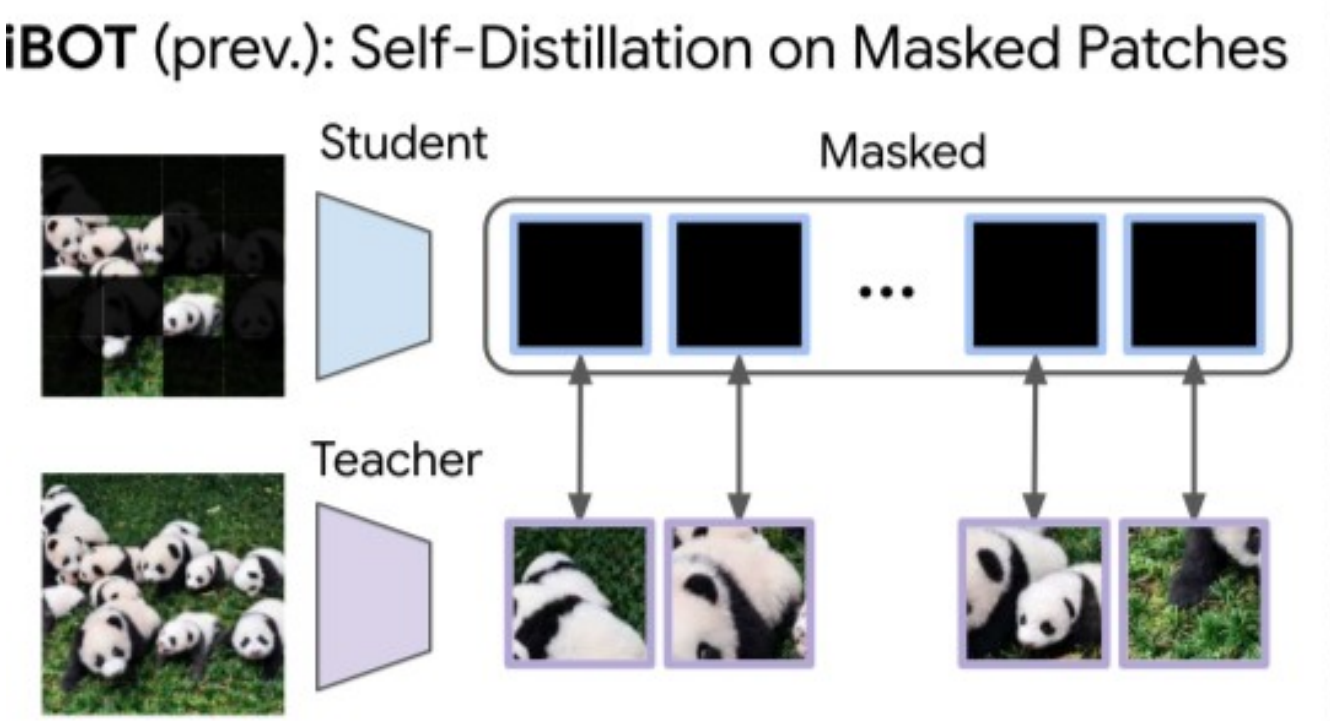
Improve the per-pixel or ”dense” features



Input | SigLIP 2 | PE Spatial | DINOv2 w/reg | DINOv3

Adding patch-level losses

- Student-teacher loss on Patch Features – “Gram Anchoring”
- And iBOT :

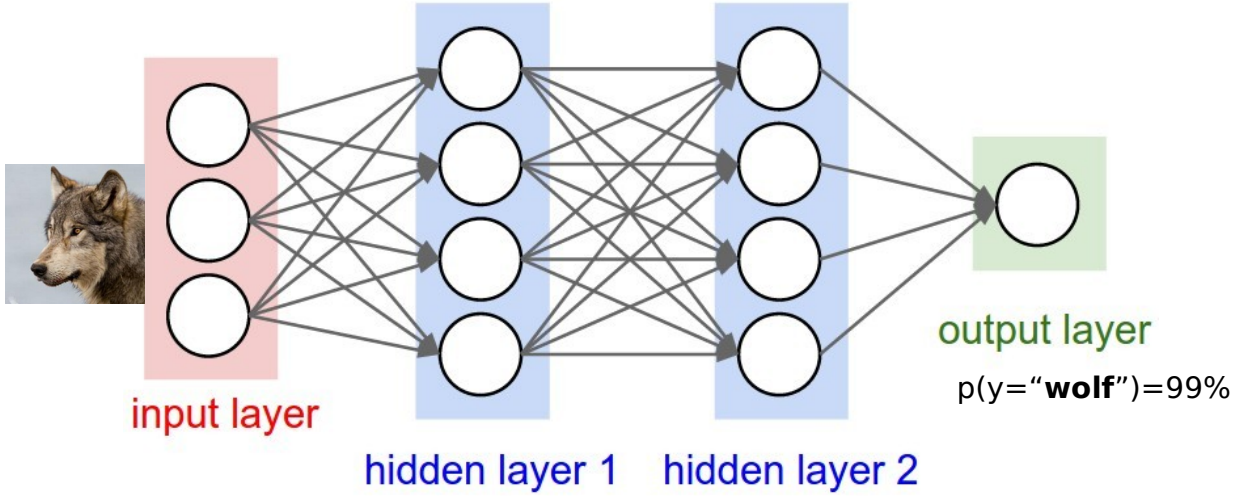


Problems with vision and uncertainty

Lead in to discussing *uncertainty*

p(y="**wolf**")=99%

p(y="**lion**")=99%

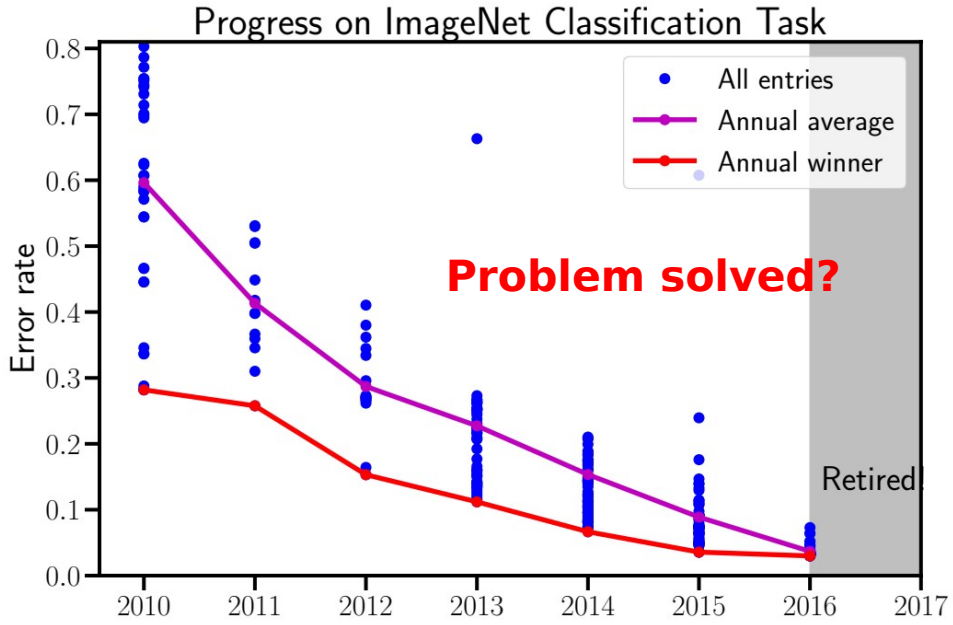


"Representation"

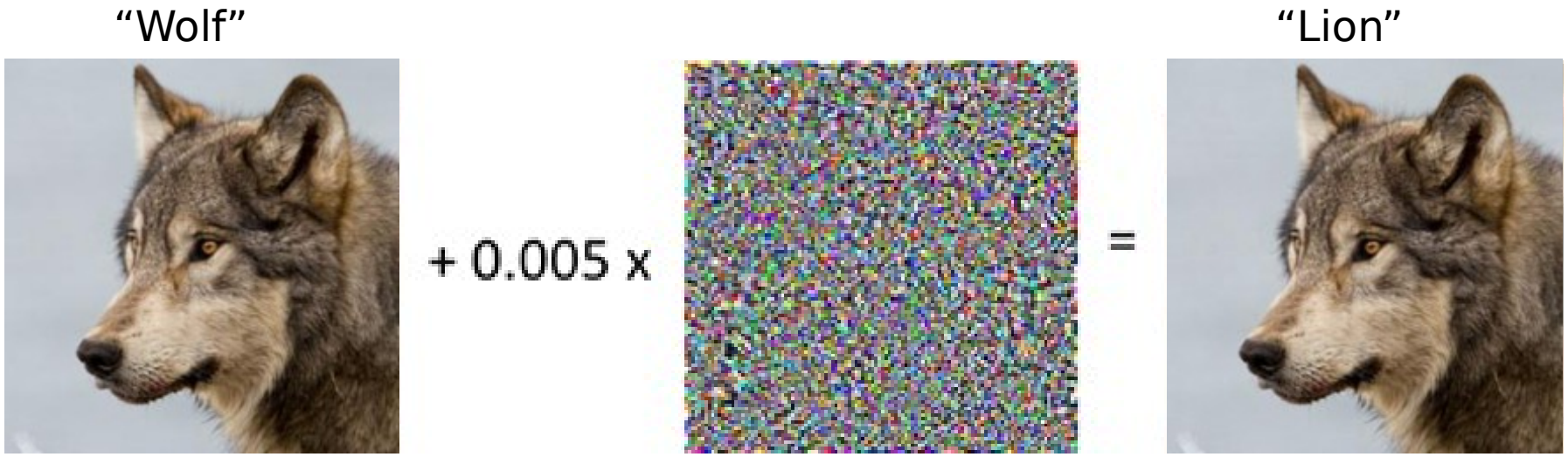
Image classification benchmarks were easy and artificial

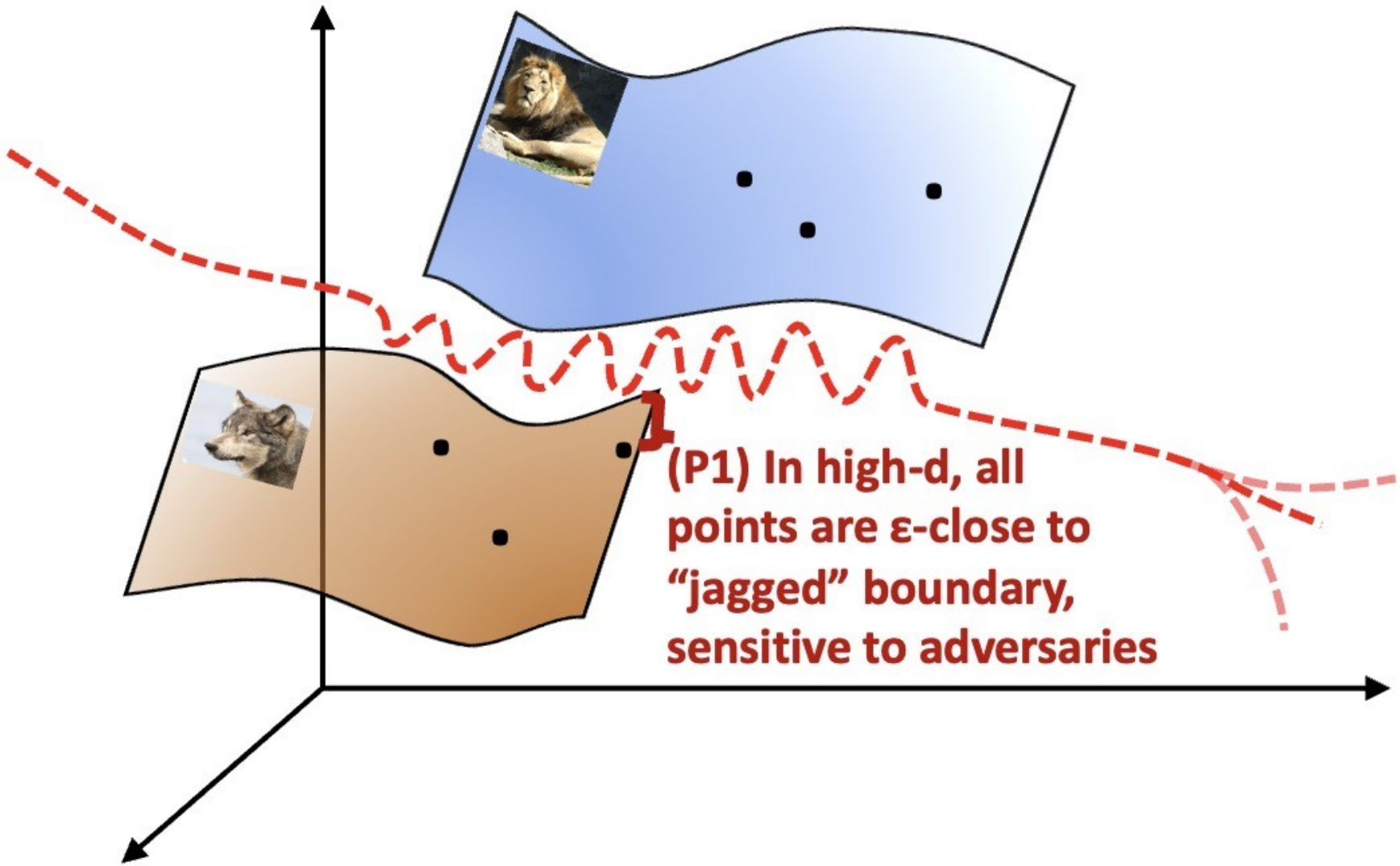
Information used by neural nets and humans is not the same!

Hard problems remain.



Sensitivity to adversarial perturbations





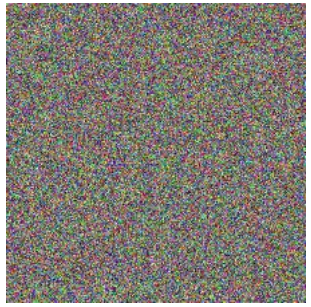
x-space (image pixels)

Real world attacks

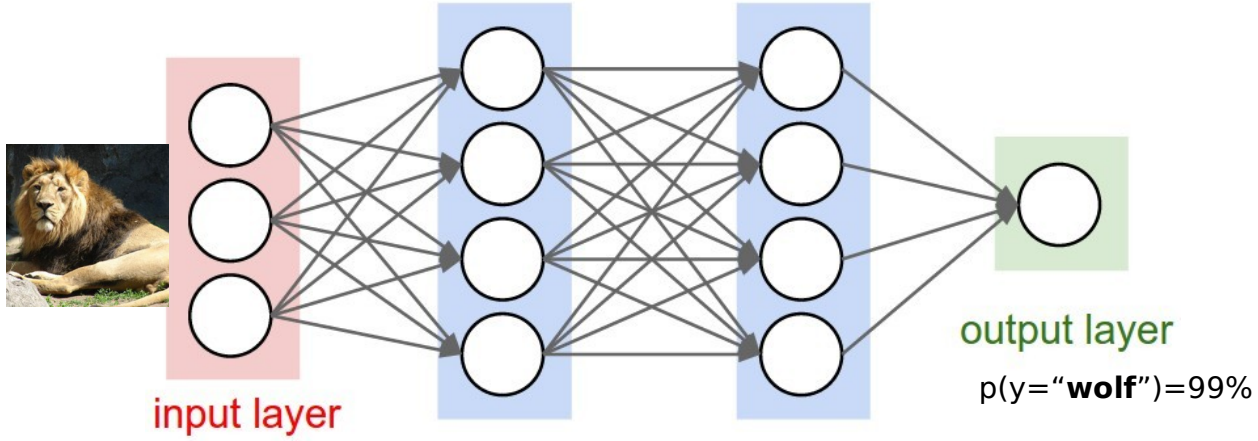
https://arxiv.org/pdf/1707.08945.pdf

Another issue – far Out Of Distribution (OOD) inputs

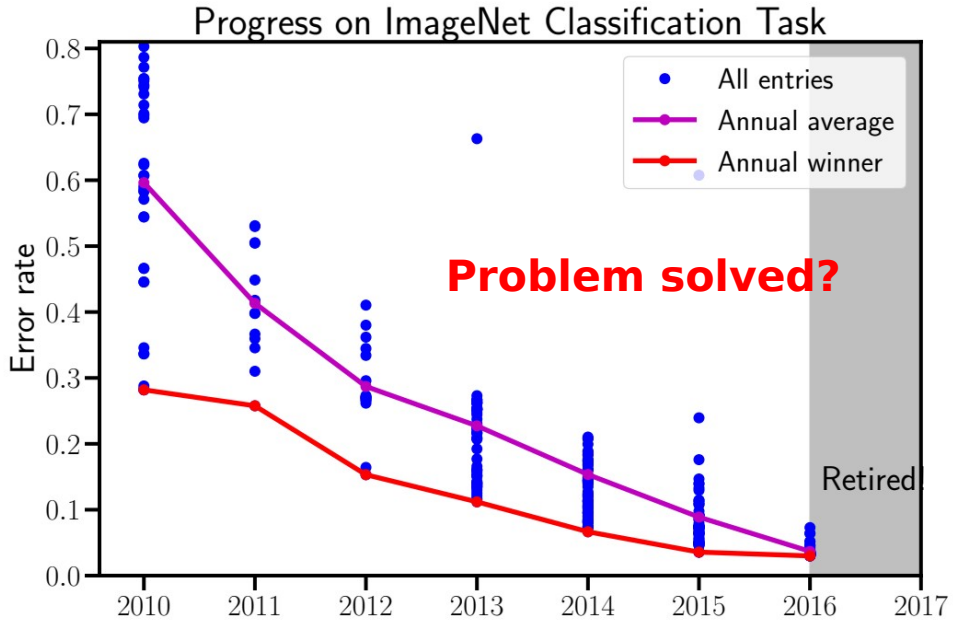
p(y=“**lion**”)=99%



p(y=“**lion**”)=99%



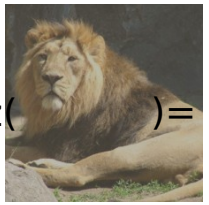
“Representation”



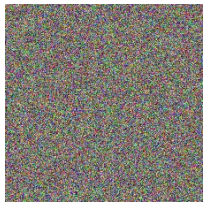
p(y="**lion**") > 99%

z( )= z( )= z( )

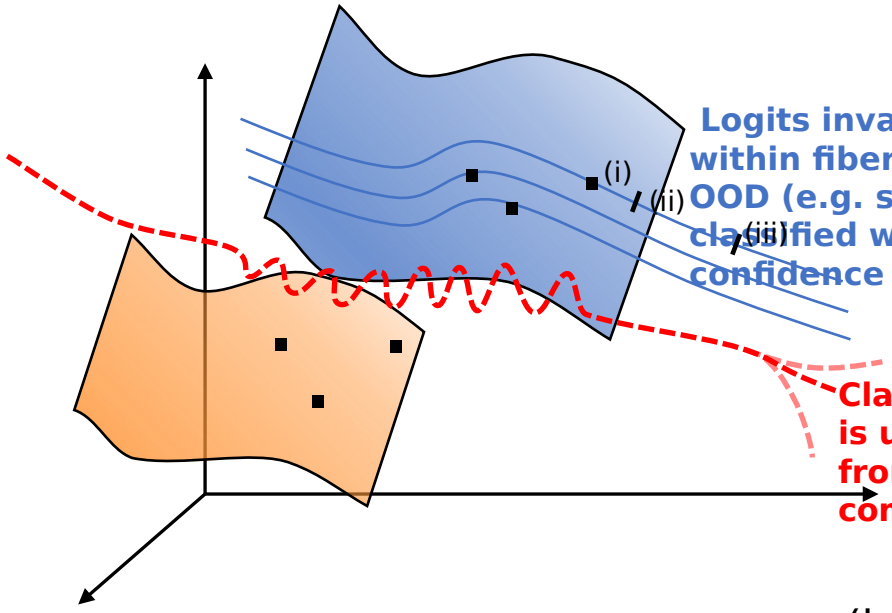
(i)



(ii)



(iii)

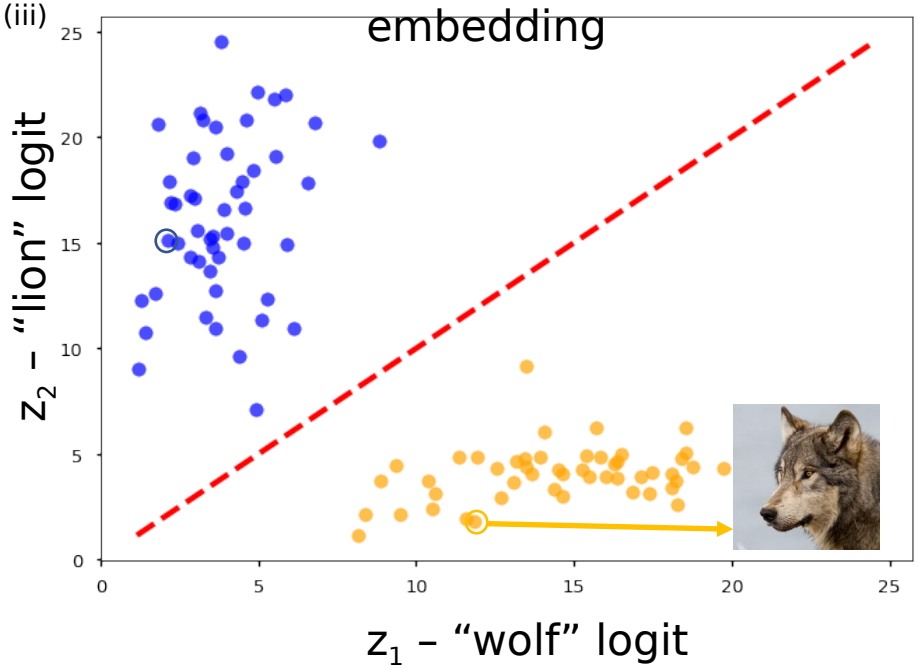


Logits invariant within fiber bundle – OOD (e.g. static) classified with high confidence

Classifier in pixel space is under-constrained far from data (e.g. image of complete noise)

x-space (image pixels)

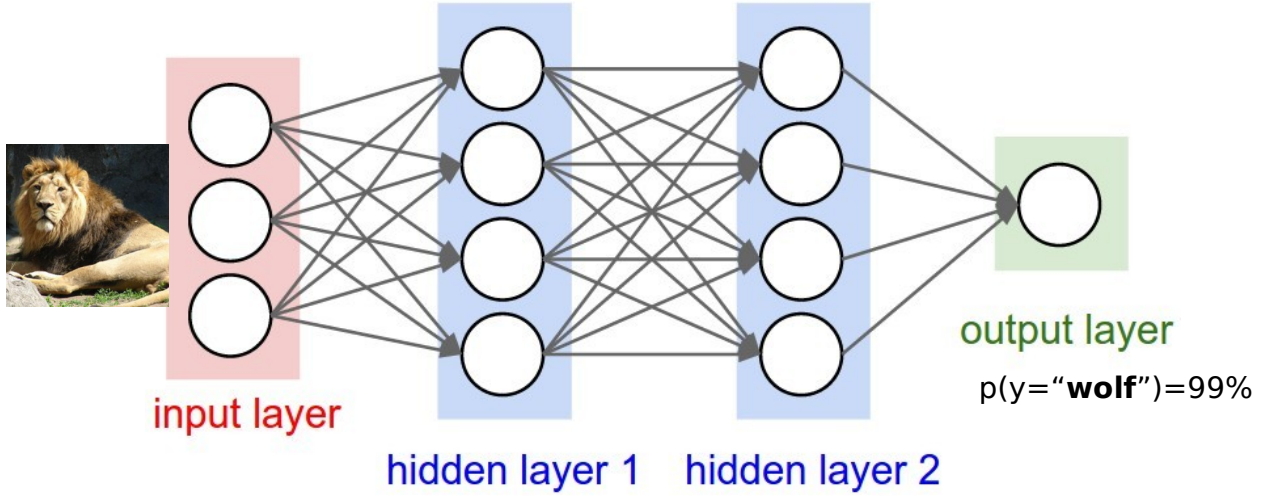
z(x) - Logit embedding



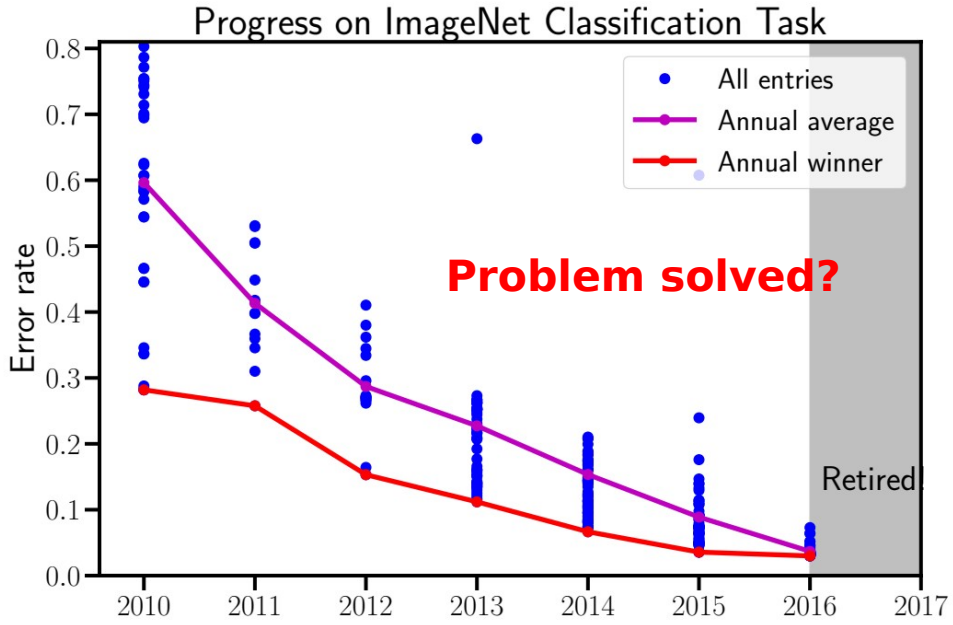
Predictions not invariant to “nuisance” variations

$\hat{y}$=“lion”

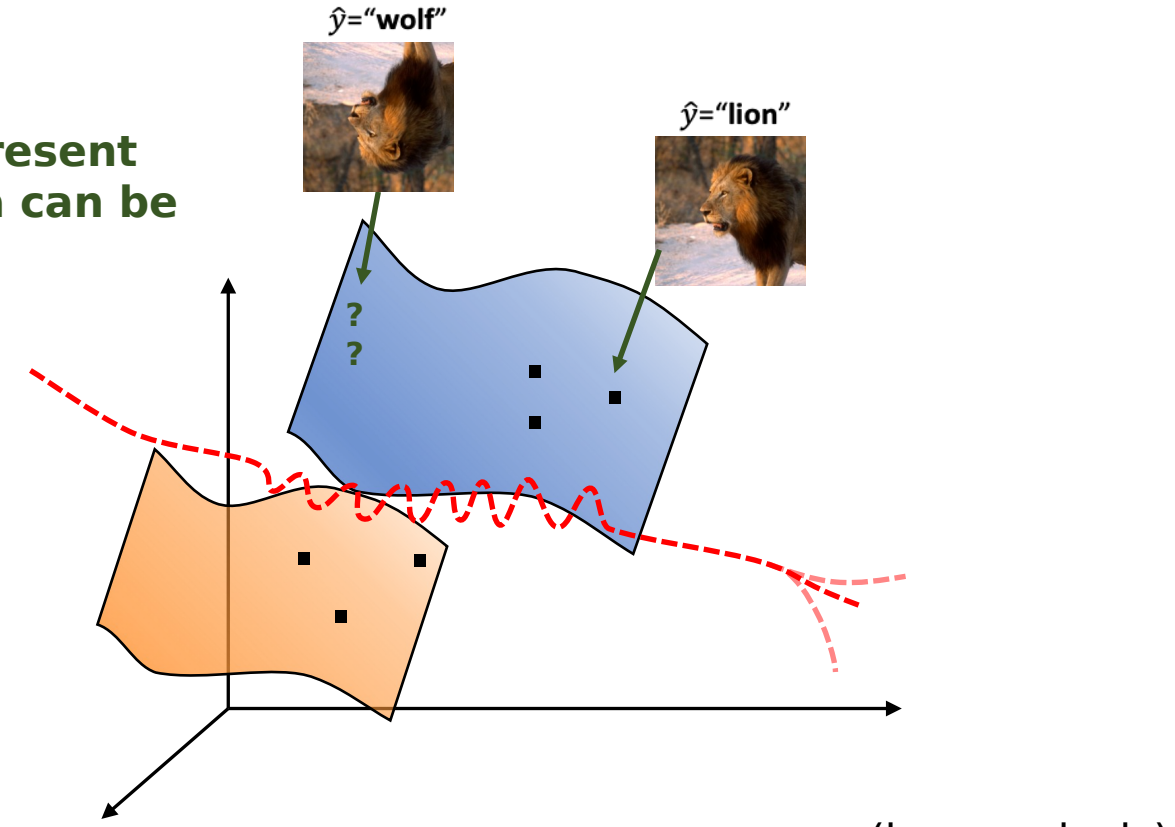
$\hat{y}$=“wolf”



“Representation”

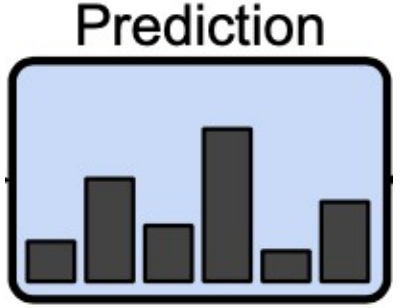


Parts of image manifold not present in training data can be misclassified

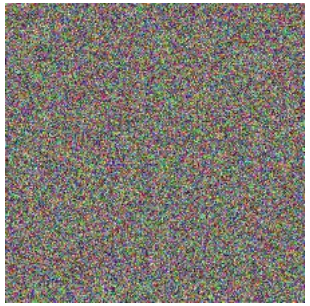


We attempt to mitigate some of these issues with semi- and self-supervised learning

What do the probabilities that our neural nets give us really *mean*?

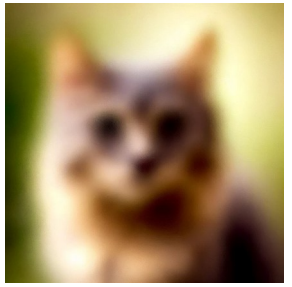


p(y="**lion**")=99%



p(y="**lion**")=99%

p(y="**lion**")=99%



p(y="**cat**")=80%

We often say that the probability a neural net assigns to the top choice is its **"confidence".** If a neural net is 80% confident of a prediction, will that prediction be correct 80% of the time?

Confidence calibration game: give a range that you think the answer lies in with probability 50%

Martin Luther King’s age at death:

Length of the Nile River:

Number of countries that are members of OPEC:

Number of books in the Old Testament:

My guesses

Martin Luther King’s age at death:

35-55

Length of the Nile River:

700-1500 miles

Number of countries that are members of OPEC:

9-23

Number of books in the Old Testament:

30-55

Correct answers

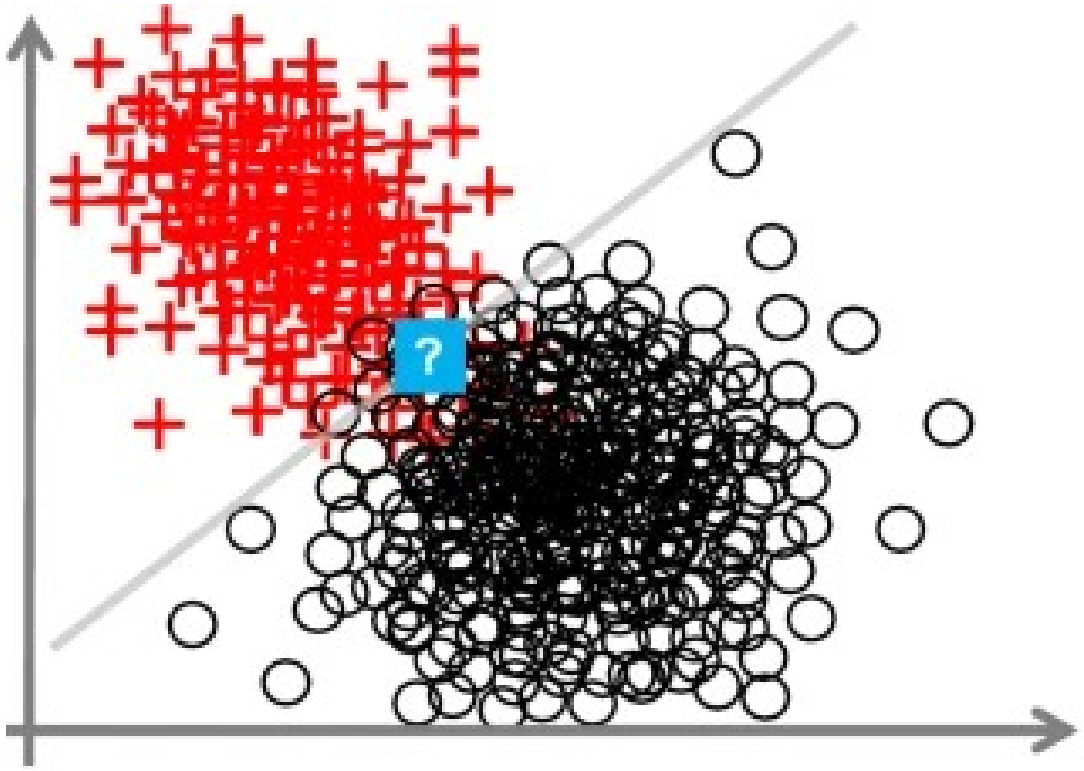
39

6737 km or 4187 m

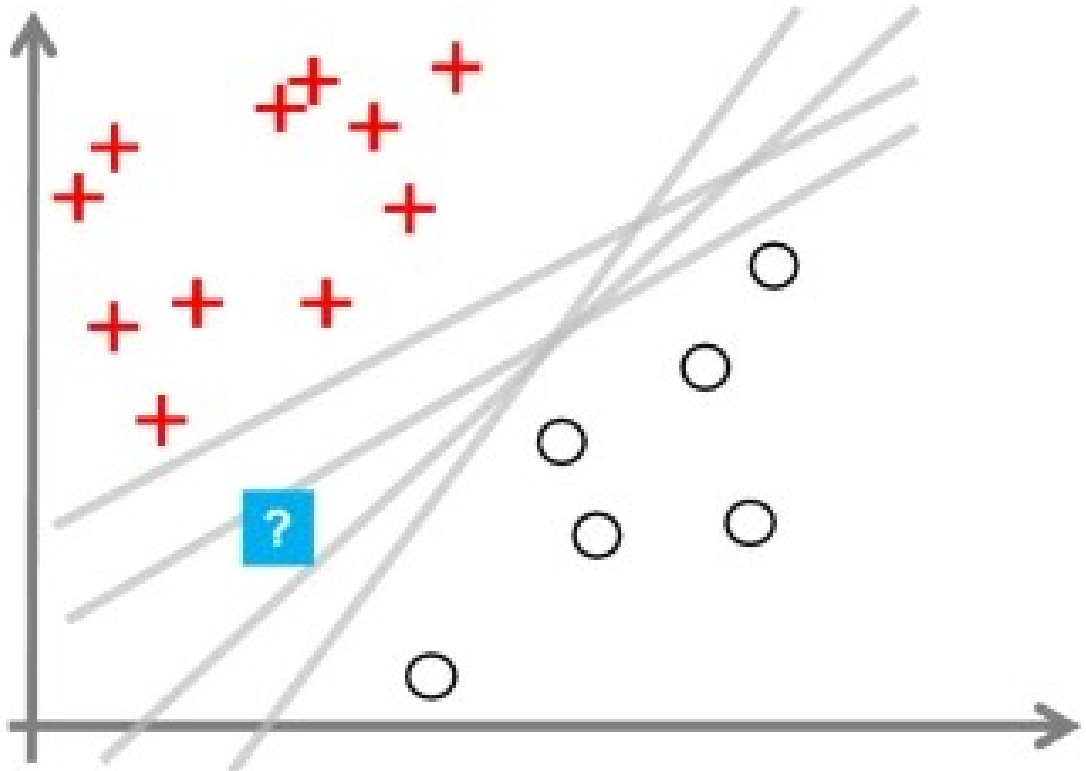
12 (as of 2026)

39

Two types of uncertainty



eatoric / fundamental / irreducible / data uncertainty



Epistemic / Model uncertainty
– due to our lack of knowledge

Do we care? Lots of applications. Right side will cause problems.